

LECTURE 1

Module 3: Random Walks (RW)

Introduction

- It is a stochastic or random process that describes a path.
- Examples of stochastic processes, pixel values in a movie => the movie itself is not random it is deterministic, however if you look at the pixel values of individual pixels in the movie frames then you'll find that they appear to be random or stochastic. Same goes for stock market graphs, chemical reactions and various everyday processes.
- A random walk is described by a succession of random steps on some mathematical space.
- Examples of mathematical spaces,
 - Real-line (R^1) & Integer-line (Z)
 - Lattice ($2D, R^2$) or (d -dimensional, R^d)
 - Graphs, Directed & Undirected

Example of a random walk on the integer line,

We start at 0 and we take steps in this walk as $a_1, a_2, a_3, \dots, a_n$

In this case, a_i can be either +1 or -1 with equal probability, i.e. we can either move one step forward or one step backward with equal probability.

Let, $a_1 = +1, a_2 = +1, a_3 = -1, a_4 = -1, a_5 = -1$

So we start at $x = 0$ and we reached, $x = 0 + 1 + 1 - 1 - 1 - 1 = -1$

This seems pretty inconsequential and doesn't provide much benefit.

But an order can emerge when the +1 and -1 steps are weighted and not equi-probable.

Say, we have probability of +1 > probability of -1, then given enough iterations for the random walk, we will get $x > 0$.

This probability is called the **transition probability** and just by varying this probability we can observe a structure develop at the macroscopic scale asymptotically and this will lead us to stationary probability distributions.

Example: Path traced by a molecule in a liquid or gas, search path of a foraging animal, price of a fluctuating stock.

In all of these cases, the movements don't make much sense in the microscopic scale but instead make sense when you look at the bigger picture, i.e. macroscopic scale.

You can even view speech as a random process at the microscopic scale and model it as a Markov chain.

Random Walk on Regular Lattice

Regular Lattice here is a 2D space where X-axis and Y-axis points are equally separated at 1 unit (so basic X-Y plane only).

A random walk on this regular lattice would be starting from point (0, 0) and moving from current point to any other point in the lattice. **But this next move should be according to some probability distribution.**

This probability distribution describes the transition probability for any two points/nodes i and j in the lattice and so the distribution will be modelled as a **transition probability matrix**.

Simple Random Walk

Location can only jump to “neighbouring” sites of the lattice forming a lattice path.

Here, the concept of neighbourhood is defined on a case-by-case basis and each neighbour can have different transition probability associated with it.

Example, considering only the 4-neighbours approach (up, down, left, right), we can put higher transition probability on the “up” neighbour as compared to the other neighbours, so even in this case, there will be a pattern emerging at the macroscopic level asymptotically.

Simple Symmetric Random Walk

When the transition probabilities are the same for all the neighbours then it's Simple Symmetric RW. So if you take 2D lattice and 8-neighbours, then each transition will have probability $1/8$.

This probability exponentially decreases as you go into the higher dimensions.

With a Simple RW, you will see that a pattern emerges at the macroscopic level asymptotically.

That is, given enough iterations for the random walk, you will find that the walk spends more time traversing certain nodes of the space/lattice/graph as compared to other nodes.

This is however quite rare to occur for a Simple Symmetric RW.

These are three instances of the same RW when at 10,000 iterations, 1 million iterations & 10 million iterations,



You can see how in the second image, dark spots begin to emerge indicating that the random walk spends more time in those parts/nodes and the spots begin to darken even more and the other areas begin to lighten even more in the third image.

And this is the convergence of a RW which gives us the stationary probability distribution.

Simple Bordered Symmetric Random Walk

State space is limited to finite time.

Transition probabilities depend on the location of the state.

That is, the transition probabilities change depending on which region the random walk is currently at.

A straightforward example of this is a square of size 3 in 2D space that's centered at (0,0). When we're at the starting point, we have equi-probable transitions to any neighbour. But as soon as we reach a bordering point like (-3, y), (3, y), (-3, -y), (3, -y), (x, 3), (-x, 3), (x, -3), (-x, -3), we cannot move in certain directions because the lattice is bounded and so transition probabilities for those neighbours are 0 in such a situation.

Random Walks as Graphs

Directed Graph $G = (V, E)$ is a collection of vertices/nodes/points V and edges E where (u, v) is an edge if vertex u is connected to (points to) vertex v .

How are RW modelled on a graph then?

A node in the graph is connected to several other nodes, so these are the neighbours of that node.

Each node will have a transition probability attached to each of its neighbours.

So in the RW,

We start from some starting node.

We pick a connected edge of that node to traverse depending on the transition probabilities.

We then traverse that edge to reach a new node where the same process repeats.

So connectivity and transition probabilities of a node play an important role in the RW.

Undirected graphs are not relevant for this course, so there's that.

Strongly Connected Directed Graphs

An ordered pair of vertices (u, v) are called "strongly connected" if a directed path leads from u to v , i.e. v is reachable from u .

A graph is said to be strongly connected when every pair of vertices (u, v) has a directed path between them, i.e. every vertex is reachable from every other vertex.

For RW on graphs, it is required that they be strongly connected for it to show any macroscopic behaviour.

Properties of RW as a graph

These properties are valid when the graph is strongly connected.

As the RW on a graph progresses (asymptotically),
The fraction of time the walk spends at various vertices in the graph “converges” to a “stationary probability”.

This stationary probability is referring to the fact that,
Once this “convergence” happens, the number of times a given node i was visited, divided by the length of the RW, i.e. the fraction of time that was spent at node i in the RW, will be a constant value which is dependent on only the value of i .
This constant value of probability can be represented as P_i

Again, this only works for a strongly connected graph. If the graph is not strongly connected, then convergence will not happen and the starting point of the RW will matter a lot in determining how the RW progresses.

Note that none of this has a bearing on the transition probabilities. The transition probability distribution is something that will be given to you from the start itself (or something that can be inferred from the graph itself, like web page linking in WWW graph) and using the transition probabilities on a strongly connected graph for a RW will converge to a stationary probability distribution where the fraction of time that the random walk spent on a given node will be constant and **dependent on only the transition probabilities** and **independent of all other factors** like starting point of RW, initial probability distribution, etc.

So all RWs on the same graph with the same transition probabilities will converge to the same stationary probability distribution irrespective of other factors like starting point.

Source and Sink nodes

Source Node: A vertex which has no incoming edge

Sink Node: A vertex which has no outgoing edge

A source node can never be reached by a RW and a RW will terminate/get stuck if it reaches a sink node.

If a graph has source or sink nodes then it would be a not so strongly connected graph.

In the context of web-graphs (where each node is a web page), edges can be visualized as a URL link from one page to the other. And sink nodes (web pages that don't link to any other page) are called dangling pages.

So in the real-world, it will be really difficult to always have strongly connected graphs and so we'll need some way to convert not so strongly connected graphs to strongly connected graphs and we'll see this later.

Strongly Connected Components (SCC)

SCCs of a directed graph partition a graph into subgraphs that are strongly connected.

SCC of a graph can be defined as,

$$\text{SCC}(G) = (V', E')$$

V' = Subset of V

E' = Subset of E that are the edges of vertices of V'

So in an SCC, every node is reachable from every other node.

Now the thing to note is that in a non-strongly connected graph, once a RW leaves an SCC, it can never return back to that SCC (think on this a bit and prove it by contradiction, so if RW did leave SCC to reach another node and eventually came back to SCC, then those nodes outside would by definition actually be inside the SCC).

LECTURE 2

Bipartite Graphs

It is a graph where the vertices can be partitioned into two sets L and R such that all edges in the graph would go from L to R and R to L only and there will be no edges from L to L or R to R .

A cycle is defined as a path of edges and vertices where a vertex is reachable from itself.

Bipartite graphs do not contain odd-length cycles because that would imply that there is an edge between two vertices in L or two vertices in R .

We start at a vertex A in L , take an edge and go to a vertex B in R .

Now if we want a cycle then we have to come back to A , we can only do that by taking the edge from B to A making it an even-length cycle.

If we decide to go to C in L , then we cannot go from C to A because edges cannot be from a vertex in L to L .

So we have to go to B or some other vertex in R and then go to A resulting in an even-length cycle again.

Another condition on Bipartite graphs is that,

$$\text{GCD}(\text{All cycle lengths}) \geq 2$$

Because all cycle lengths have to be even and all even numbers will have GCD at least 2.

The scope of how stationary probability distribution behaves for random walk on bipartite graph is out of the syllabus but it is pretty cool :P

Stationary Probability Distribution

We have a directed graph with N nodes and we start a RW from vertex i .

And we define the starting/initial probability distribution by placing a mass on the vertex i as,

$$p_i = 1$$

$$p_j = 0 \text{ if } j \neq i$$

This can be represented in vector form as a row vector,

$$\begin{aligned}\underline{P} &= [p_1 \quad p_2 \quad \dots \quad p_i \quad \dots \quad p_N] \\ &= [0 \quad 0 \quad \dots \quad 1 \quad \dots \quad 0]\end{aligned}$$

[Do note that this is different from the transition probability distribution.]

In the general case we can start with any vertex and not just vertex i , so we'll fill the initial probability distribution with arbitrary values ≥ 0 for all vertices.

But what does this arbitrary probability distribution mean? That is, what does it mean to have a RW start at vertex i with probability p_i ?

It will make more sense when we take an ensemble of realizations.

What is a realization?

It can be described as one "instance" of a RW, i.e. one run of the RW from starting vertex to ending vertex.

Example, measuring voltage fluctuations in a circuit for 1 minute followed by 1 minute of break. Each measurement interval is a separate realization of a RW.

Multiple realizations of the RW make an ensemble of the random process.

Now this statement makes sense

"what does it mean to have a RW start at vertex i with probability p_i ?"

p_i is the fraction of times the starting node of the RW is chosen to be node i .

Let M be the number of realizations and n_i be the number of realizations whose starting vertex is i .

Then,

$$\sum_{i=1}^N n_i = M$$

So we can say that,

$$\frac{n_i}{M} = \tilde{p}_i \rightarrow p_i \text{ as } M \rightarrow \infty [\text{asymptotically}]$$

This is just putting into words what we have described earlier.

So picking the starting vertex can be described as,

Tossing an N sided biased dice where the bias of the dice is described by the initial probability distribution $\underline{P}$.

Similarly for each move made in the RW, we'll have a new probability distribution.

How to move to another vertex in RW?

Consider initial probability distribution to be the following,

$$\underline{P}^{(1)} = [p_1 \quad p_2 \quad \dots \quad p_i \quad \dots \quad p_N]$$

Suppose that we're at some vertex i at time t.

Then, what is the probability that we're at vertex j at time t + 1? That is, what is the probability that our next move in the RW is to vertex j?

$$p_j^{(t+1)} = \sum_{\forall i \text{ connected to } j} p_i^{(t)} \cdot p_{ij} \implies (1)$$

$$p_j^{(t+1)} \implies \text{Probability of being at vertex } j \text{ at time } t+1$$

$$p_i^{(t)} \implies \text{Probability of being at vertex } i \text{ at time } t$$

$$p_{ij} \implies \text{Transition probability of going from vertex } i \text{ to vertex } j$$

$p_i^{(t)}$ is a recursive function that has base case as, $p_i^{(1)} = p_i$ [setting $t = 1$]

If we're at vertex j at time t + 1 then we can be at any of the connected neighbours i (edge from i to j) at time t which is why we include all neighbours i while computing $p_j^{(t+1)}$.

We can also rewrite equation 1 as,

$$p_j^{(t+1)} = \sum_{i=1}^N p_i^{(t)} \cdot p_{ij} \implies (2)$$

We go from 1 to N in the summation because the p_{ij} value will be 0 if there is no edge from i to j, so the condition "i connected to j" over the summation will be implicitly handled like this.

Relation between probability distributions at time t and time t + 1,

$$\underline{P}^{(t)} = \begin{bmatrix} p_1^{(t)} & p_2^{(t)} & \dots & p_i^{(t)} & \dots & p_N^{(t)} \end{bmatrix} \implies (3)$$

The probability $p_i^{(t)}$ represents the probability of being at vertex i at time t.

Similarly,

$$\underline{P}^{(t+1)} = \begin{bmatrix} p_1^{(t+1)} & \dots & p_i^{(t+1)} & \dots & p_j^{(t+1)} & \dots & p_N^{(t+1)} \end{bmatrix} \implies (4)$$

Once we take the next step at time t from node i to reach a new node j at time $t + 1$, we have a new probability distribution which is what $\underline{P}^{(t+1)}$ represents.

Using equations 3 and 4, we can rewrite the general form of equation 2 in matrix form as,

$$\underline{P}^{(t+1)} = \underline{P}^{(t)} \cdot P$$

Where,

P = State Transition Probability Matrix

P_{ij} = Probability of RW at vertex i selecting the edge $i \rightarrow j$ to reach vertex j

Note that the sum of a row i in the transition probability matrix has to be exactly 1.

And if you remember this from MML part 1, this is the definition of a row stochastic matrix (we had column stochastic matrix in MML part 1).

To prove that this equation does give what we want, we can open up the vectors and matrix P and see what happens,

$$\begin{bmatrix} p_1^{(t+1)} & p_2^{(t+1)} & \dots & p_j^{(t+1)} & \dots & p_N^{(t+1)} \end{bmatrix} = \begin{bmatrix} p_1^{(t)} & p_2^{(t)} & \dots & p_i^{(t)} & \dots & p_N^{(t)} \end{bmatrix} \cdot \begin{bmatrix} p_{11} & p_{12} & \dots & p_{1j} & \dots & p_{1N} \\ \dots & \dots & \dots & p_{2j} & \dots & p_{2N} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ p_{i1} & p_{i2} & \dots & p_{ij} & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ p_{N1} & p_{N2} & \dots & p_{Nj} & \dots & p_{NN} \end{bmatrix}$$

And we can get equation 2 properly now as,

$$p_j^{(t+1)} = \begin{bmatrix} p_1^{(t)} & p_2^{(t)} & \dots & p_i^{(t)} & \dots & p_N^{(t)} \end{bmatrix} \cdot \begin{bmatrix} p_{1j} \\ p_{2j} \\ \dots \\ p_{ij} \\ \dots \\ p_{Nj} \end{bmatrix}$$

$$p_j^{(t+1)} = \sum_{i=1}^N p_i^{(t)} \cdot p_{ij}$$

Both are the same.

So the matrix form equation is also valid.

Note: Matrix P is not symmetric because the probability of transitioning from i to j may not be the same as j to i . Diagonal entries of P may be non-zero as well because a node can link to itself.

Fundamental Property of RW

In the limit, the long-term probability of being at a particular vertex is “independent” of the start vertex or an initial probability distribution provided the underlying graph is strongly connected.

$$\lim_{t \rightarrow \infty} p_j(t) \rightarrow \pi_j \text{ [Constant value that is independent of initial probability distribution]}$$

This is the limiting probability which is the stationary probability distribution and denoted by π .

$$\pi \implies [\pi_1 \quad \pi_2 \quad \dots \quad \pi_i \quad \dots \quad \pi_N]$$

The stationary probability distribution should be such that taking more steps in the RW from this probability distribution would give us the same probability distribution back.

That is,

$$\underline{P}^{(t+1)} = \underline{P}^{(t)} \cdot P \implies \underline{P}^{(t+1)} = \underline{P}^{(t)}$$

We can rewrite it using π as,

$$\pi = \pi \cdot P$$

This basically tells us that the only thing the stationary probability distribution π is dependent on is the transition probability distribution P .

Important thing to note is that π is the left eigenvector of the P matrix with eigenvalue = 1

$$x \cdot A = \lambda \cdot x$$

In this case,

$$\pi \cdot P = 1 \cdot \pi$$

P is a row stochastic matrix which you should remember from MML part 1 that it will have a dominant eigenvalue which will be 1 and power iteration can be used here to find π which will be the dominant left eigenvector of P .

LECTURE 3

Interpretation of the following equation using Trellis visualization,

$$p_j^{(t+1)} = \sum_{i=1}^N p_i^{(t)} \cdot p_{ij}$$

Also proving the same equation using $n_i^{(t+1)}$ and stuff.

If you understand the intuition and interpretation behind this equation then there's nothing else of note here. The Trellis visualization is just for aiding the intuition and nothing that we have to remember as such.

LECTURE 4

Showing how the RW movement equation works with the help of Total Probability Theorem (TPT). Again nothing to remember as such, it's just for intuition's sake.

Markov Chain (MC)

For our discussion, we can consider Markov Chains to be equivalent to RWs but MCs have subtle differences attached to them related to higher order statistics that we need not be concerned with.

A Markov Chain consists of a finite set of states and there are transitions between any two states.

For each pair of states (x, y) , there is a transition probability p_{xy} for transiting from x to y such that,

$$\text{for each } x, \sum_y p_{xy} = 1$$

So this makes P a row-stochastic matrix (x is on the row and y are the columns) just like the P matrix that was discussed for RWs.

A RW in a MC starts at some state (following some initial probability distribution).

At a time step, if it is in state x , then the next state y is randomly selected with probability p_{xy}

And the transition probability matrix P would just be filled with these elements as,

$$P(x, y) = p_{xy} \text{ for all } x \text{ and } y$$

One more way of describing a MC,

A MC is a directed graph where each vertex is a state and each edge from vertex x to vertex y has weight p_{xy}

The MC is connected if the underlying directed graph is strongly connected, i.e. if there is a directed path from every vertex to every other vertex.

What is this underlying directed graph?

The transition probability matrix P induces a graph on the Markov Chain.

So basically just remember that $p_{xy} = 0$, if vertices x and y are not adjacent to each other and p_{xy} will be some value > 0 and ≤ 1 if x and y are adjacent.

So the values of the P matrix are implicitly creating a graph that the Markov Chain will follow.

Sidenote: Ravi Kannan's proof for convergence of RW to π

[No need to remember any of this as such, it's just an alternate way to tackle our proof.]

We had defined π to be the probability at time step t which is asymptotic.

However in Kannan's book, he takes the average/expected probability over time t

So the equation of convergence remains the same,

$$\pi = \pi \cdot P$$

But the definition of π has changed as,

$$\pi^{(t)} = \frac{1}{t} [\underline{p}^{(1)} + \underline{p}^{(2)} + \dots + \underline{p}^{(t)}]$$

So to get the stationary probability distribution π , we find the average of all probability distributions upto that time step t .

Convergence of RW to a Stationary Probability Distribution

Statement: If the probability distribution of the RW converges to a stationary probability distribution then for some time step t , the probability distribution at that instant should be the same as the distribution at the previous time instant.

$$\text{That is, } p^{(t+1)} = p^{(t)} \cdot P \implies p^{(t+1)} = p^{(t)}$$

We set this to be π , so, $\pi = \pi \cdot P$

And clearly, π is the left eigenvector of the transition matrix P with corresponding eigenvalue = 1.

[Recall,

$$A \cdot x = \lambda \cdot x \implies x = \text{right eigenvector}$$

$$x \cdot A = x \cdot \lambda \implies x = \text{left eigenvector}]$$

Perron-Frobenius Theorem (P-F theorem)

It talks about the properties of the leading/dominant eigenvalue and the corresponding eigenvector of a non-negative real square matrix.

We won't prove this theorem, just use the concept of the P-F eigenvalue and r-Perron root

[Transition matrix P satisfies these constraints]

Theorem:

For a non-negative, real, square matrix A , $\exists$ a real number r called the r-Perron root, such that the Perron-Frobenius (P-F) eigenvalue $r > |\lambda|$ where, λ are the other eigenvalues of A (possibly complex)

So r will be the leading eigenvalue of the matrix A .

What is the implication of the P-F theorem on RW?

If a MC is “irreducible” & “aperiodic” then the largest eigenvalue of the transition matrix P will be 1 and all other eigenvalues will be strictly smaller than 1.

That is, the r-Perron root of P is 1.
The proof of this is not discussed here.

Irreducible MC: A MC is irreducible if the state-graph is strongly connected

State Graph

This is just the underlying graph of the MC which was informally described before but a formal definition of this graph is as follows.

A MC, C , has a state set, V , (assume $|V| = N$ which is finite) and a transition matrix P where P_{ij} = Probability of going from state i to state j

Let G_C be state graph of C such that,

- The nodes of G_C are V
- G_C contains a directed edge (i, j) if $P_{ij} > 0$

We can also state irreducibility as: an MC is irreducible if each pair of states of the MC inter-communicates with each other where state i communicates with state j if state j is reachable from state i and state i inter-communicates with state j when both are reachable from each other.

IMPORTANT NOTE for Strongly Connected-ness of P :

A hole in the P matrix is an entry where $P_{ij} = 0$.

If we allow for holes in the P matrix, then the state graph can still be strongly connected.

However such a graph can quickly become non-SC when new nodes are added and checking for strongly connected-ness is quite difficult in such a scenario (because the algorithm itself is quite time-consuming).

In order to avoid such situations, we impose the condition that all entries in P must be greater than zero, so the graph is guaranteed to be always strongly connected because every node is directly connected to every other node.

Aperiodic MC:

$\text{GCD}(\text{All Cycle Lengths in } G_C)$ is called the period of G_C .

A MC is aperiodic if the $\text{GCD}(\text{All Cycle Lengths in } G_C) = 1$ or the period of $G_C = 1$

Basically, the State Graph has to be atleast non-bipartite because $\text{GCD}(\text{All Cycle Lengths in Bipartite Graph}) \geq 2$.

The period of a state S_i in given MC is defined as the GCD of all cycle lengths where the starting and ending state is S_i

Regular Matrix

A stochastic matrix for which some power of it has all positive entries is a regular matrix.

In our case, if an MC is aperiodic and irreducible, then the transition probability matrix of the MC has to be a regular matrix.

Just a fact that can be useful for answering questions in exams and nothing else :(

Using the Power Iteration Method to find dominant eigenvector

Start with some initial probability distribution, $\underline{P}^{(0)}$ at $t = 0$

$$\text{At } t=1, \underline{P}^{(1)} = \underline{P}^{(0)} \cdot P$$

$$\text{At } t=2, \underline{P}^{(2)} = \underline{P}^{(1)} \cdot P = \underline{P}^{(0)} \cdot P^2$$

...

$$\text{At } t=t, \underline{P}^{(t)} = \underline{P}^{(t-1)} \cdot P = \underline{P}^{(0)} \cdot P^t \implies (1)$$

Stop when $\underline{P}^{(t)} = \underline{P}^{(t-1)}$

P is an NxN stochastic matrix

Let the left eigenvectors of P be,

$$[v_0 \quad v_1 \quad \dots \quad v_{N-1}]$$

P is a diagonalizable matrix, i.e. we can write P as, $P = U \cdot D \cdot U^{-1}$ (think eigen decomposition).

Why is it diagonalizable => Not discussed here

But because it is diagonalizable, it has n distinct eigenvalues and n distinct eigenvectors that are independent.

Therefore its eigenvectors are a basis vector for $\mathbb{R}^N$ and span $\mathbb{R}^N$ space and because the initial probability distribution $\underline{P}^{(0)}$ is an N-dimensional vector, it can be written as the linear combination of the eigenvectors of P.

Therefore,

$$\underline{P}^{(0)} = c_0 v_0 + c_1 v_1 + \dots + c_{N-1} v_{N-1} \implies (2)$$

We can use equation (2) in (1) to get,

$$\underline{P}^{(t)} = \left(\sum_{k=0}^{N-1} c_k \cdot v_k \right) \cdot P^t \implies (3)$$

We can take P^t inside the summation to get,

$$\underline{P}^{(t)} = c_0 \cdot v_0 \cdot P^t + \dots + c_{N-1} \cdot v_{N-1} \cdot P^t$$

Now we can use the fact that v vectors are the left eigenvectors of P.

$$\text{So, } v_k \cdot P = v_k \cdot \lambda_k$$

Where, λ_k is the k^{th} eigenvalue associated with v_k

Now we can write equation 3 as,

$$\underline{P}^{(t)} = \sum_{k=0}^{N-1} c_k \cdot v_k \cdot (\lambda_k)^t \implies (4)$$

Why can we do this?

Because we can also define eigenvectors as,

Applying matrix A on a vector x, gives us a vector with a new direction y.

However if x is the eigenvector of A, then applying A on x would give us a vector in the same direction just scaled by a factor given by its eigenvalue.

So if we keep applying matrix A n number of times on vector x, we would still get the same direction vector (x) in the output, just the eigenvalue will be scaled by the specified number of times, i.e. $A^n \cdot x = \lambda^n \cdot x$

Now we can expand equation 4 as,

$$\begin{aligned} \underline{P}^{(t)} &= c_0 v_0 (\lambda_0)^t + c_1 v_1 (\lambda_1)^t + \dots + c_{N-1} v_{N-1} (\lambda_{N-1})^t \\ &= \lambda_0^t \left[c_0 v_0 + \sum_{k=1}^{N-1} c_k \cdot v_k \cdot \left(\frac{\lambda_k}{\lambda_0} \right)^t \right] \implies (5) \end{aligned}$$

But since we have the P-F theorem applied to the MC which is irreducible (since $P_{ij} > 0$) and aperiodic (no bipartite graph), it follows that,

$$\lambda_0 = 1 \text{ and } \lambda_1, \dots, \lambda_{N-1} < \lambda_0$$

$$\lambda_0 > \lambda_k$$

$$\frac{\lambda_k}{\lambda_0} < 1$$

$$\therefore \left(\frac{\lambda_k}{\lambda_0} \right)^t \rightarrow 0, \text{ as } t \rightarrow \infty$$

Therefore we can write equation 5 as,

$$\lim_{t \rightarrow \infty} \underline{P}^{(t)} = c_0 v_0 \implies (6)$$

That is, $\underline{P}^{(t)}$ converges to the dominant/leading eigenvector associated with the eigenvalue 1 within a multiplicative constant c_0

It is also proved that $c_0 = 1$ for positive stochastic matrices but that's not discussed here (quite involved proof that is discussed later).

We also have,

$$v_o = v_o \cdot P \implies (7)$$

$\therefore v_o$ is the leading left eigenvector of P with $\lambda = 1$

We know that,

$$\begin{aligned} \underline{P}^{(t+1)} &= \underline{P}^{(t)} \cdot P \\ &= c_0 v_0 \cdot P \implies [\text{from (6)}] \\ &= c_0 \cdot v_o \implies [\text{from (7)}] \\ \underline{P}^{(t+1)} &= \underline{P}^{(t)} = \underline{\pi} \end{aligned}$$

So finally we get,

$$\underline{\pi} = \underline{\pi} \cdot P$$

And because $c_0 = 1$, we can say that, $\underline{\pi} = v_0$

So as t becomes asymptotic, the contribution of the other left eigenvectors of P diminishes to such an extent that it can be described as noise and only the leading eigenvector actually contributes something.

Obviously when t is not very large, all the eigenvectors contribute to the probability distribution $P^{(t)}$.

Note: If the transition probability matrix does not have a full set of independent eigenvectors, then we can do the same calculations on the Jordan-Canonical form of the matrix but this is out of scope of the syllabus.

LECTURE 5

Google PageRank

World Wide Web (WWW) viewed as a MC

The web is viewed as a directed graph where each web page is a vertex and each edge corresponds to a hypertext link from one page to another.

We want to rank these web pages and the overall conclusion is that the rank will be assigned according to the stationary probability of the web pages which makes sense because this just tells us that the rank of a page will be dependent on the fraction of time a random walk (or user) visited a particular web page after running for an extended period of time (asymptotically).

Every edge (link) from page (node) i to page (node) j has a transition probability P_{ij} associated with it. How to determine this?

$$P_{ij} = \frac{1}{c(i)}$$

Where, $c(i)$ = Outdegree of node i , i.e. number of links pointed to by page i

Size of P is $N \times N$ where N is the number of web pages in WWW.

So every page j linked to page i has equal probability of being visited via transition $i \rightarrow j$

Suppose there is a vertex i which connects to two vertices j and k , then, $c(i) = 2$

Then, $P_{ij} = \frac{1}{2}$ and $P_{ik} = \frac{1}{2}$. Similarly transition probabilities will be determined for each vertex i wrt each outgoing vertex j to fill the i^{th} row of P matrix.

If webpage i is connected to every webpage, then $P_{ij} = 1/N$.

If webpage i is connected to no webpage (dangling node), then $c(i) = 0$ and so $P_{ij} = 0$ for all j

[Just FYI, ofcourse the WWW graph will not be strongly connected and there will be loads of problems related to the MC stationary probability convergence properties that we have discussed till now.]

PageRank Definition

[PageRank has a recursive definition]

Suppose we want to compute the PageRank for node A .

We had already stated that **PageRank for a node is its probability in the stationary probability distribution for the given transition probability matrix.**

Let, $\text{PageRank}(A) = r(A)$ and using the original equation that we had with MC we can write this as the following, where T_i is any possible neighbour node and we're transitioning from T_i to A .

$$r(A) = \sum_{i=1}^N r(T_i) \cdot P(A|T_i)$$

From T_i we can go to any of its neighbours with equal probability which is related to its outdegree as,

$$1/c(T_i)$$

$$\therefore P(A|T_i) = \frac{1}{c(T_i)}$$

$$\therefore r(A) = \sum_{i=1}^N \frac{r(T_i)}{c(T_i)}$$

So in order to know the page rank for node A , we have to know the page rank for all the other nodes in the graph.

This forms an equation for PageRank of A and like this we will have 1 equation for each node in the graph, i.e. N equations.

So we have N unknowns and N equations, so we can solve this using $AX = B$ matrix formulation but for large N (for the original Google paper, $N = 24$ million) this is not practical so it's not of much use and we'll use RW properties instead.

So now to actually compute the PageRanks we have two approaches,

1. Start a RW with an initial probability distribution and let it run for a long amount of time for the probability distribution to resolve to the stationary probability distribution which would represent the PageRanks for each page.
2. Compute the leading/dominant left eigenvector of the transition probability matrix P which would also return the stationary probability distribution / PageRanks.

Problem with finding PageRank:

The stationary probability convergence only exists if the graph of the MC is strongly connected (P-F theorem result).

The WWW graph however is not strongly connected.

So the RW may stop at a dangling node or it may never be able to enter a source node (which has no incoming links).

Solution (one approach): Restart Condition

This method basically involves filling in all the 0 entries in the P matrix by a small number which is essentially the floor value for the matrix entries.

We can formally describe this method as...

At each time step t at vertex T_i do one of the following,

- a. With probability α (**restart probability**) jump/teleport to any vertex selected uniformly at random. So transitioning to any vertex will have probability $1 / N$ and any entry in the transition probability matrix P has to have value atleast α/N
- b. With probability $1 - \alpha$ select an outgoing edge from T_i and follow it.

In the Google paper, $\alpha = 0.15$, so 15% of the time the RW will randomly teleport to any of the N pages in the WWW graph and 85% of the time the RW will transition to an adjacent page with probability dictated by the transition probability matrix P (we have assumed uniform probability wrt outdegree of node).

Note: We teleport with equal probability α/N .

If we are at a dangling node, we will continue iterating till we get (a) possibility, i.e. teleportation.

Let G be the adjacency matrix of the graph where the rows have been normalized to add up to 1. Normalisation is done by dividing each entry in a row by the sum of that row, basically just dividing $G_{ij} / c(i)$, G is row stochastic matrix and called the static transition probability matrix, basically this is how adjacency matrix G is converted to the standard transition probability matrix that we know, P .

(Sir continued to use the notation of calling it G from now on, so that's why I'm going ahead with it as well).

[Note: Lot of implementation details are discussed in the "Solved Example" section]

Therefore we can say that the actual probability of going to an adjacent node, i.e. (b) possibility, is given by,

$$(1 - \alpha) \cdot \underline{\pi} \cdot G$$

where, $\underline{\pi}$ = Static stationary probability distribution of G

Why is this so?

Let $\underline{P}^{(t)}$ be the probability distribution at time t

and let $\underline{P}^{(t+1)}$ be the probability distribution at time $t+1$

then,

$$\underline{P}^{(t+1)} = \underline{P}^{(t)} \cdot G$$

But at $t \rightarrow \infty$, $\underline{P}^{(t+1)} = \underline{P}^{(t)} = \underline{\pi}$

And we consider this option with probability $(1 - \alpha)$

So we get overall probability of not going to teleportation as,

$$(1 - \alpha) \cdot \underline{\pi} \cdot G$$

The problems with considering G matrix is that it can have zero entries as well, source nodes will have a column of zeros and dangling nodes will have a row of zeros or some nodes i and j can just not be directly connected.

This violates the condition that we imposed (all entries in G must be positive) to apply the P-F theorem.

To avoid this we can combine (a) & (b) possibilities into one equation to get the stationary probability distribution as,

$$\underline{\pi} = \alpha(1/N, 1/N, \dots, 1/N) + (1 - \alpha) \cdot \underline{\pi} \cdot G$$

$$\underline{\pi} = \frac{\alpha}{N}(1, 1, \dots, 1) + (1 - \alpha) \cdot \underline{\pi} \cdot G \implies (1)$$

And now there will be no zero entries in the stationary probability distribution.

So, $\underline{\pi} = \underline{\pi} \cdot G$ has become equation (1)

[This will become much more clear in the Solved Example section]

And now we can solve for π_i in this equation because we know G and N and we assume the value of α .

Can verify equation (1) via dimensionality analysis as well,

$$(1, 1, \dots, 1) \implies 1 \times N$$

$$\underline{\pi} \implies 1 \times N$$

$$G \implies N \times N$$

We can write equation (1) in plain English as,

This is by the definition of PageRank $\underline{\pi}$ being the stationary probability distribution associated with a RW with transition probability matrix G modified with restart/teleportation (wrt probability α).

We can rearrange equation (1) as follows,

$$\pi[I - (1 - \alpha)G] = \frac{\alpha}{N}(1, 1, \dots, 1)$$

$$\pi = \frac{\alpha}{N}(1, 1, \dots, 1) \cdot [I - (1 - \alpha)G]^{-1}$$

So in principle we can find the closed form solution for π_i by computing, $[I - (1 - \alpha)G]^{-1}$
 But this is finding the inverse of an $N \times N$ matrix and N as we know can be very large (24 million at the least). So this solution is not very practical.

What is more practical is that we can literally run the RW on the graph governed by (G, α) and determine the time step where the probability distribution converges to the stationary probability distribution π_i

Another way to do this is to use the power iteration method to find the leading eigenvector of G which has to be π_i as we have discussed before.

Running the RW literally is the least computationally expensive option whereas computing the inverse is the most expensive option.

[There are more implementation details not discussed for now, but they will be cleared in the tutorial section, i.e. solved example]

Personalized PageRank

Equation (1) can be written as,

$$\pi = \frac{\alpha}{N} \underline{r} + (1 - \alpha) \cdot \pi \cdot G$$

where, $\underline{r} = (1, 1, \dots, 1)$

Because $\underline{r}$ is filled with 1s we have uniform probability distribution for teleportation.

However we can replace it by a **non-uniform probability distribution (preference vector) $\underline{s}$ that is specific to a user.**

$$\pi = \frac{\alpha}{N} \underline{s} + (1 - \alpha) \cdot \pi \cdot G$$

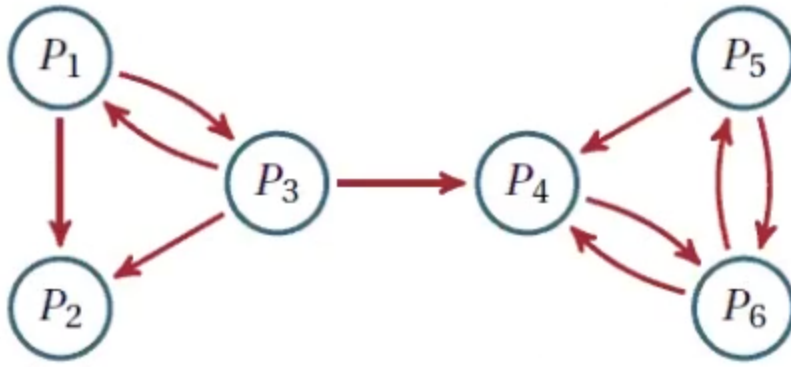
So this is how we can compute Personalized PageRanks for each user.

In the original Google paper, the WWW graph would be stored every day and RW would be run on the stored graph. So the graph is not dynamically changing every minute or anything like that (atleast back in 1998 :()

LECTURE 6

Solved Example for PageRank on a sample graph

Consider the following graph,



The adjacency matrix G for this graph looks like,

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

We know that the Page Rank for a page P_i is given as,

$$r(P_i) = \sum_{P_j \in L_{P_i}} \frac{r(P_j)}{c(P_j)}$$

where, L_{P_i} = Set of pages that link to P_i

So using this in our example graph we get 1 PageRank equation for each page as follows,

$$r(P_1) = \frac{r(P_3)}{3} \implies \text{Only } P_3 \text{ is incoming to } P_1 \text{ \& } c(P_3) = 3$$

$$r(P_2) = \frac{r(P_1)}{2} + \frac{r(P_3)}{3}$$

$$r(P_3) = \frac{r(P_1)}{2}$$

$$r(P_4) = \frac{r(P_3)}{3} + \frac{r(P_5)}{2} + \frac{r(P_6)}{2}$$

$$r(P_5) = \frac{r(P_6)}{2}$$

$$r(P_6) = \frac{r(P_4)}{1} + \frac{r(P_5)}{2}$$

So we want to find the PageRank row vector,

$$v = [r(P_1) \quad r(P_2) \quad r(P_3) \quad r(P_4) \quad r(P_5) \quad r(P_6)]$$

Since $r(P_i)$ represents the proportion of time that our RW spends at page P_i , the vector v should be a probability vector (PMF). So each $P_i \geq 0$ and the sum of vector v has to be equal to 1.

We now **row-normalize the adjacency matrix G** by dividing each element by the sum of its row, this gives us new G matrix which is row-stochastic (except for dangling node rows) as,

$$G = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1/3 & 0 & 1/3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{bmatrix}$$

Now we express our stationary probability distribution equation as,

$$v = v \cdot G \text{ (same as, } \underline{p_i} = \underline{p_i} \cdot P \text{)}$$

$$v = [r(P_1) \quad r(P_2) \quad r(P_3) \quad r(P_4) \quad r(P_5) \quad r(P_6)] \cdot \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1/3 & 0 & 1/3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{bmatrix}$$

Now we will use the power iteration method to find the leading eigenvector of G which would be vector v .

For vector v , we start with uniform initial probability distribution, $v^{(0)}$ which is given as,

$$v^{(0)} = \left[\frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \right]$$

And now we start iteration as,

$$v^{(1)} = v^{(0)} \cdot G$$

$$v^{(2)} = v^{(1)} \cdot G$$

...

But according to the P-F theorem this iteration will not converge v to a stationary probability distribution because the graph G is not strongly connected.

The interesting thing is that iterating on even this graph G would result in convergence of vector v . In this solved example, in around 25 iterations, v converged to the following vector,

$$\left[0 \quad 0 \quad 0 \quad \frac{1}{5} \quad \frac{2}{15} \quad \frac{4}{15} \right]$$

But the problem with this, if you have noticed, is that the sum of converged vector v is $3/5$ whereas it should have been 1 because vector v is supposed to represent a PMF.

This problem is caused by the dangling node which in this example is P_2 .

To overcome this, we use the teleportation logic.

However, unlike what was discussed before, we'll define teleportation in a different way. Teleportation is only allowed for dangling nodes and we implement this logic in the G matrix by setting all elements in the dangling node's row to $1/N$.

So the new G' matrix is given as,

$$G' = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 & 0 & 0 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 1/3 & 1/3 & 0 & 1/3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{bmatrix}$$

And G' is a fully row-stochastic matrix.

Now we can again start iteration for G' with the same $v^{(0)}$ initial vector and we find that it also converges to a vector in around 25 iterations which in this case would be,

$$v = [0 \quad 0 \quad 0 \quad 1/3 \quad 2/9 \quad 4/9]$$

And this vector actually does sum up to 1 which means that it represents a PMF.

So we have satisfied all our conditions, but what's the problem then?

There are 3 zeros which means that the PageRank for those pages are 0 which doesn't make sense, **any page must have non-zero PageRank.**

Why did we get page ranks for the first 3 pages as zero?

Because those 3 vertices form a separate SCC of the graph and once you leave this SCC you can never return back to it and that is what actually happened in the iteration (RW) as well which is why their PageRank is zero.

How to overcome this problem?

We will use the restart probability concept, in this example we use damping factor, d , which is equal to $1 - \alpha$.

With probability $(1-d)$ we'll teleport to any of the N pages and with probability d we'll continue transitioning to a linked page.

[So a non-dangling node can also teleport in this scenario]

We can combine these conditions to replace each row of the matrix G' , r , as,

$$r = dr + (1 - d) \left[\frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \quad \frac{1}{6} \right]$$

And this would give us a new matrix G (**Google Matrix**) as,

$$G = d \cdot \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix} + (1 - d) \cdot \begin{bmatrix} \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \end{bmatrix}$$

In the original Google paper, $d = 0.85$ and so we get G for our graph as,

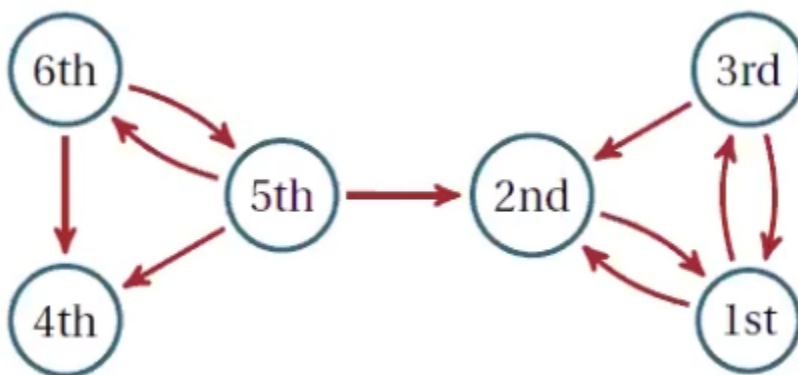
$$G = \begin{bmatrix} \frac{1}{40} & \frac{9}{20} & \frac{9}{20} & \frac{1}{40} & \frac{1}{40} & \frac{1}{40} \\ \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} & \frac{1}{6} \\ \frac{37}{120} & \frac{37}{120} & \frac{1}{40} & \frac{37}{120} & \frac{1}{40} & \frac{1}{40} \\ \frac{1}{40} & \frac{1}{40} & \frac{1}{40} & \frac{1}{40} & \frac{1}{40} & \frac{7}{8} \\ \frac{1}{40} & \frac{1}{40} & \frac{1}{40} & \frac{9}{20} & \frac{1}{40} & \frac{9}{20} \\ \frac{1}{40} & \frac{1}{40} & \frac{1}{40} & \frac{9}{20} & \frac{9}{20} & \frac{1}{40} \end{bmatrix}$$

And this matrix has no holes in it and so it is guaranteed that G represents a strongly connected graph and G is a positive row-stochastic matrix.

And running RW on this G matrix and $v^{(0)}$ vector converges to stationary probability distribution v which in this case is,

$$v = [0.0517 \quad 0.0737 \quad 0.0574 \quad 0.2800 \quad 0.1851 \quad 0.3521]$$

And this is how the PageRank for each page looks like,



The convergence happens with some error margin, in case of 50 to 100 iterations we get accurate PageRanks upto 3 to 7 digits and this is how many iterations Google actually runs the RW for, so they guarantee atleast that much accuracy for the PageRanks.

So the sum of the converged vector might not equal to 1 with some error margin of some digits.

You can try starting with a different starting vector $v^{(0)}$ but you'll find that it too converges to the same PageRanks as shown above.

LECTURE 7

Intro to MCMC Sampling

MCMC = Monte Carlo Markov Chain Sampling

We are given a pmf/pdf for a variable $\underline{x}$ in d-dimensional space as, $p(\underline{x})$ [written formally as, $f_X(\underline{x})$]

$$\underline{x} = [x_1 \quad x_2 \quad \dots \quad \dots \quad x_d]^T$$

We want to generate samples that follow this pdf.

$\underline{x}$ can be a continuous or discrete variable.

We have already seen the notion of generating a random variable that follows a certain PMF/PDF.

In the case of random walks, we had two significant PMFs,

1. The initial probability distribution with which the first vertex of the RW was selected so the random variable is the first vertex of the RW.
2. Each row of the transition probability matrix (TPT) is also a PMF which is satisfied by the RW so in this case one random variable is associated with each vertex in the graph which counts how many times that vertex was visited in a RW which obviously will follow that vertex's PMF/TPT row.

In these situations we never actually created or dealt with any random variables but instead just tossed an n-sided coin that was biased according to the corresponding PMF and this ensured that the RW followed these PMFs appropriately.

Now we will deal with the problem of generating random variables for real.

“Generate samples that follow a given PMF” means that if you calculate the PMF of the generated samples, then it would be approximately the same as the given PMF from which the samples were generated.

Parameter Estimation

You are given 2D data points where data can belong to two classes. Consider this to be the same as training data for supervised learning.

For predicting whether a new point belongs to class 1 or class 2, in the maximum likelihood estimation technique, you have to estimate the PMF of the two classes (fit two Gaussians to data belonging to each class separately) and determine to which class is the new point more likely to belong to.

The PMFs (Mean and Covariance matrix of the two Gaussians) are the parameters that we are estimating.

In this case, the supervised training data points are the same as samples and so in this problem, we are given the samples and we are using it to estimate the PMFs associated with it.

So we can say that Parameter Estimation is the inverse of what we want to do, which is finding samples from a given PMF, in this analogy it would mean, given the mean and covariance matrix for a Gaussian we want to generate data points that fall in that Gaussian.

Applications:

1. We can use this generation of samples to generate synthetic data for ML tasks which can help with the training process.

2. RBM Parameter Estimation (advanced topic so no explanation as such).

3. Calculation of marginal probabilities (individual dimension's probabilities)

Normally we calculate marginal probabilities by removing the unwanted variables from the pdf/pmf as,

If $\underline{x}$ is discrete,

$$p(x_1) = \sum_{x_2, \dots, x_d} p(\underline{x})$$

If x is continuous,

$$p(x_1) = \int_{x_2, \dots, x_d} p(\underline{x}) dx$$

But analytically solving these equations can be very expensive when we talk about higher dimensional space, especially the continuous case, as a d -dimensional $\underline{x}$ will require $d-1$ integrations to compute just $p(x_1)$.

Suppose that we have data of height and weight of people. We can plot this on the X-Y plane with height on x_1 -axis and weight on x_2 -axis.

The PMF, $P(x_1 = X, x_2 = Y)$, tells us the probability of a person having height X and weight Y . This is represented as a 2D histogram where (X, Y) would determine a histogram bin and the Z-axis value corresponding to the (X, Y) bin would indicate that bin's frequency/probability mass.

$$\sum_{l=1}^N \sum_{k=1}^N p_{1,2}(k, l) = 1$$

The marginal probabilities in this distribution would be,

$$P(x_1 = X) \text{ \& } P(x_2 = Y)$$

That is, what is the probability that a person would have height = X regardless of their weight and what is the probability that a person would have weight = Y regardless of their height.

These marginal probabilities are also PMFs because,

$$\sum_{k=1}^N p_1(k) = 1 \text{ \& } \sum_{l=1}^N p_2(l) = 1$$

and are represented as a 1D histogram where the X-axis would represent the height/weight and the Y-axis would represent the frequency/probability mass of that height/weight.

We can calculate these marginal probabilities as,

$$p_2(l) = \sum_{k=1}^N p(k, l)$$

For a given weight, iterate through all the heights

$$p_1(k) = \sum_{l=1}^N p(k, l)$$

For a given height, iterate through all the weights

And if these were continuous variables, we would just replace summation with integration and enjoy that mess.

It is possible that we're given the PDF of x in functional form (so not a literal distribution) and it can easily happen that integrating this PDF in functional form to calculate marginal probabilities is really difficult.

So how can sampling help here?

We can use the given PDF in functional form to generate n samples and fill the d -dimensional space to get a PMF and now we can sum over this distribution directly.

Basically, say you are given heights and weights of people as, $p(x, y) = \log(x) + \log(y)$
So instead of integrating over this complicated PDF, we can find n number of points/samples (x, y) that follow this PDF (i.e., samples) and then compute $p(x)$ and $p(y)$ using just summation by considering these samples as representative of the entire PDF.

Note:

N = No. of entries in the population (actual data, N = infinite in case of continuous data)

n = No. of entries in the sampled data (n is chosen by us).

4. Find Expectation of $f(\underline{x})$ over $p(\underline{x})$

Given an arbitrary function $f(\underline{x})$, we want to find its expected value over a PMF/PDF $p(\underline{x})$.

We can find this analytically for the discrete and continuous cases as,

$$E(\underline{f}) = \sum_{x_1, x_2, \dots, x_d} f(x_1, x_2, \dots, x_d) \cdot p(x_1, x_2, \dots, x_d)$$

$$E(\underline{f}) = \int_{\underline{x}} f(\underline{x}) \cdot p(\underline{x}) d\mathbf{x}$$

But just as finding the marginal probabilities can become difficult in the integration case, **finding the expected value can also be difficult in the integration case**, either because $f(\underline{x})$ or $p(\underline{x})$ is analytically uninteractable (integral is difficult).

So we can overcome this problem as,

Sample the function $f(\underline{x})$ such that it follows PDF $p(\underline{x})$ to get n samples and then just find the average of the n samples to get an approximate answer to $E(\underline{f})$.

Basically, **compute the sample mean**.

Link to ML: Empirical Risk Optimization

The loss function in this will be analogous to $f(\underline{x})$ and we'll have a PDF associated with it, so we won't use integration and instead compute the sample mean.

There are many more applications using sampling from a PMF/PDF but this is the main one to discuss.

Formal Definition for Sampling

Obtain a set of samples,

$$\{\underline{x}^{(l)}, l = 1, \dots, L\}$$

In $\mathbb{R}^d$ space drawn independently from the distribution $p(\underline{x})$

We can find expectation of $f(\underline{x})$ over $p(\underline{x})$ as,

$$\bar{f} = \frac{1}{L} \sum_{l=1}^L f(\underline{x}^{(l)})$$

Note: This equation doesn't contain $p(\underline{x})$ because it is absorbed by the action of summing up over all the samples which are drawn from $p(\underline{x})$.

This represents one sample/one realization of $E(\underline{f})$

We can do a large number of realizations/samples, compute their sample means and we can be pretty confident that this **mean of sample means converges to the actual function's expected value**.

$$E(\bar{f}) = E(f)$$

Sampling Methods

1. Using CDF

Sampling from standard distribution using the Cumulative Distribution Function (CDF)

The CDF for random variable $\underline{x}$ is defined as,

$$F(\underline{x}) = \int_{-\infty}^x p(z) \cdot dz = p(-\infty \leq z \leq x)$$

And we can get the PDF from the CDF as,

$$p(x) = \frac{\partial F(x)}{\partial x}$$

All CDFs look the same graphically, it starts from 0 at -infinity and increases according to the rate of change of the PDF and eventually approaches to 1 as x tends to infinity.

If the PDF represents a Gaussian distribution then we can describe the CDF curve in more detail as... It starts rising gradually from 0 at -infinity from an almost plateau like surface and it changes rapidly at a particular inflection point after which it again starts plateauing to 1 at +infinity.

Let, $y = F(x)$

Therefore, $x = F^{-1}(y)$

Let u be a random variable such that, $u < y$ that maps to the random variable z such that $z < x$.

Therefore, $z = F^{-1}(u)$

$$-\infty \leq z \leq x \ \& \ 0 \leq u \leq y$$

We can use this to reach the following conclusion,

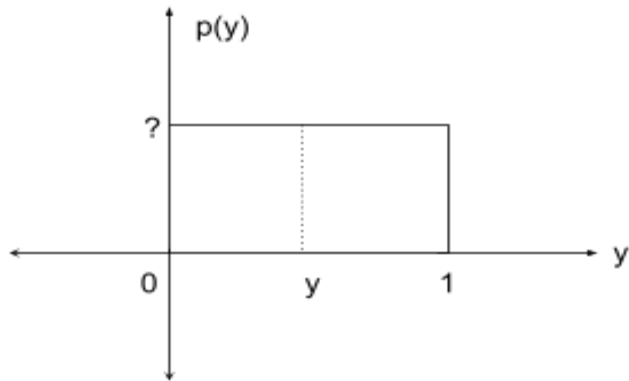
$$\begin{aligned} p(u < y) &= p(z < x) \\ &= F(x) \end{aligned}$$

$$p(u < y) = y$$

And this property must be satisfied at each point u in range (0, 1)

[Because y can take values in range(0, 1) as it represents probability]

So the PDF of y has to be a uniform distribution and gives us the following figure for the PDF of u ,



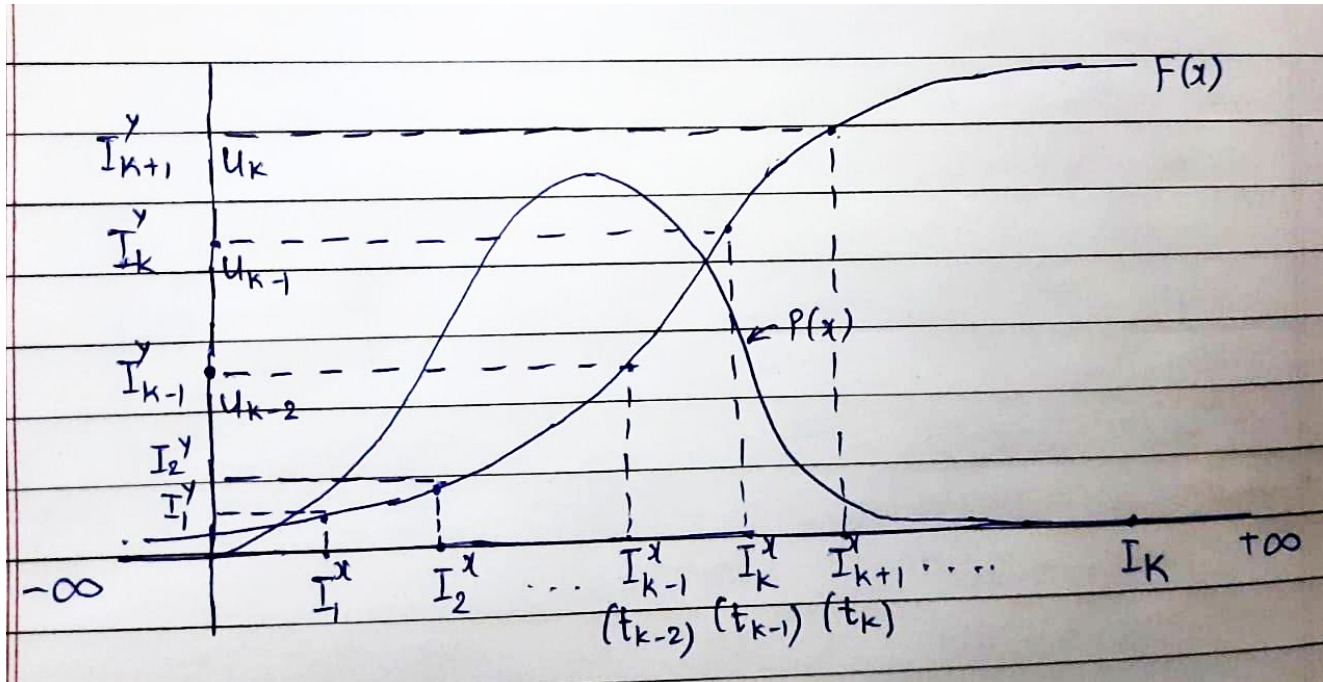
We know that the property, $p(u < y) = y$ must hold true throughout this figure.

So the area under the curve at any point y [rectangle with bottom-right corner at $(y, 0)$] has to be y which can only be possible if the height of PDF is 1, i.e. $? = 1$

And you can see that the area of the entire rectangle is $1 * 1 = 1 \Rightarrow$ So $\text{sum}(\text{PDF}) = 1$, i.e. $p(u < 1) = 1$ and $p(u < y) = y$

We now do the following constructions,

- Have a PDF for random variable x running from $-\infty$ to $+\infty$.
- We divide the X-axis into K equally spaced intervals, $I_1^x, I_2^x, \dots, I_k^x, \dots, I_K^x$ (superscript x because intervals represent X-axis) with t_k representing the “tick” value for each interval, i.e. the value of the right-end of the corresponding interval.
- For a sufficiently large K value, this discrete PMF would be a good approximation of the given PDF.
- We then plot the CDF of the given PDF.
- We then project the tick value of each interval onto the CDF and project it onto the Y-axis, so now we get K intervals on the Y-axis as well $I_1^y, I_2^y, \dots, I_k^y, \dots, I_K^y$ and let the tick values for these intervals be, $u_1, u_2, \dots, u_k, \dots, u_K$
- Because of how the CDF curve is shaped, the intervals on the Y-axis which are closer to 0 and the intervals which are closer to 1 will appear very narrow/thin in comparison to the intervals which are closer to the so called inflection point of the CDF which will have much wider/fatter intervals. Do note that the intervals are evenly spaced on the X-axis.
- This can be better understood by an image but I doubt how good the below image is,



Let x_k be a point in interval I_k that represents the probability mass for that interval (so all the probability mass of that interval is located at x_k).

Then the probability mass of interval I_k can be given as,

$$\begin{aligned} p(x_k) &= F(t_k) - F(t_{k-1}) \\ &= p(t_{k-1} \leq x_k \leq t_k) \end{aligned}$$

We also know that,

$$P(u < y) = F(x) = y$$

So we can use this result to rewrite PMF of I_k as,

$$\begin{aligned} p(x_k) &= F(t_k) - F(t_{k-1}) \\ &= p(u < u_k) - p(u < u_{k-1}) \end{aligned}$$

Because we can write, $p(u < y) = y$, we can also write $p(u < u_k) = u_k$

Therefore,

$$p(x_k) = u_k - u_{k-1} = I_k^y$$

So the size of the k^{th} interval on Y-axis determines the probability mass of the k^{th} interval on X-axis, i.e. x_k

What is the implication of any of this?

- Generate a random number, u from a uniform distribution in range(0, 1) using a pseudo random number generator [we assume that we know how to do this already].
- Let u represent the height in the Y-axis.
- Slide along the X-axis from point $(0, u)$ until you cut $F(x)$, i.e. CDF of x .

- Drop down from this intersection point to the X-axis to get x . This can be described in a equation directly as, $x = F^{-1}(u)$
- This gives us a number x which follows $p(\underline{x})$, i.e. we got a random number generator for x which follows the distribution $p(\underline{x})$ using a random number generator for range (0, 1) which follows uniform distribution.

Problems

- **You need to be able to compute $F^{-1}(u)$** , it can easily happen that this inverse function is not analytically computable in which case we're stuck again. This is why CDF is very rarely used for Sampling and why we prefer to go for MCMC sampling.
- Another problem is that this method **only works for 1D distribution sampling**, i.e., x belongs to $\mathbb{R}^1$ and cannot be used for higher dimensional sampling.

Why doesn't this method work for higher dimensions?

Because the intersection of the random number generated u and the CDF plot is a single plot in 1D sampling.

But for higher dimensions, CDF would cover n dimensional space and u would be a hyperplane (of $n-1$ dimensions), so the intersection of u and CDF will not be a single point but rather a series of points (quite possibly infinite set of points) and so there is no clear answer to $x = F^{-1}(u)$.

Summary

First we have a continuous PDF which we discretize into k intervals to get a PMF (histogram).

We then want to sample from a multinomial distribution (histogram) with k possible outcomes with associated probabilities $\theta_1, \theta_2, \dots, \theta_k$ (we already know these theta values because we made the PMF)

We want to reduce the problem of sampling a multinomial variable to the problem of sampling a single uniform variable by subdividing a unit interval into k regions with region i having size θ_i

Then we sample uniformly from $[0,1]$ and return the value of the region (value from 1 to k) in which our sample falls.

So basically we mapped each of the k PMF values of the multinomial distribution into the uniform distribution such that the length of each of the intervals in the uniform distribution would be equal to the value of corresponding PMF in the multinomial distribution.

So when we pick a number between 0 and 1 uniformly, we are automatically going to be picking 1 out of the k intervals and each interval only has length equal to its PMF in the multinomial distribution, so this distribution is followed properly.

LECTURE 8

2. MCMC Sampling

Overview

What we have at the start is a distribution $p(\underline{x})$ that we want to sample from.
We'll discretize this PDF to become a PMF according to some criteria.

Now we connect it to the concept of convergence of RW.
We divide/discretize the n-dimensional space into a lattice of grid points, i.e. vertices which the RW can visit.

We'll set $p(\underline{x})$ to be equal to the stationary probability distribution of the RW, i.e.
 $\pi = p(\underline{x})$

Now we have to solve the equation,
 $\pi = \pi \cdot P$, where $P = \text{TPM}$

So we are going to use the stationary probability distribution to determine the transition probability matrix (TPM), which is the reverse of what we did before.

Say we do this somehow.

Now we have a transition probability matrix P that follows π and we know that $\pi = p(\underline{x})$

Now we start a RW from any starting vertex (grid point) and let it converge to π .

This means that all the grid points that have been visited in this RW follow the probability distribution π , i.e. $p(\underline{x})$

Because remember what π represents, π_i represents the fraction of time the RW visited vertex i (some grid point) and this basically means that the probability that vertex i will be present in any RW is $\pi_i = p(x_i)$.

So the points are actually the samples that we wanted from $p(\underline{x})$

And we're done!

[Note: Initial probability distribution has no use here as any starting vertex works]

And you can also see how this method is independent of the number of dimensions of x . Even if the dimensions are higher, we'll again divide the space into grid points and discretize the PDF into PMF and continue with the concepts of RW convergence which are also applicable for higher dimensions. So it generalizes way better than the CDF method.

Metropolis-Hasting Algorithm

Setup for the Algorithm

This algorithm will be used to determine the TPM for a given π .

For $\underline{x} \in R^d$

$$\underline{x} = [x_1 \quad x_2 \quad \dots \quad x_d]$$

Let $p(\underline{x})$ be the PDF

Consider the discrete case when each component x_k takes a value from the set $\{1, 2, \dots, n\}$

So each component (dimension) of $\underline{x}$ can only take n values.

Note that the value that $\underline{x}$ takes isn't in range $(1, n)$, each dimension of the R^d space is divided into n intervals and so the value that a particular x_i takes can be obtained by dividing max value of that dimension by n .

If $d = 1$, then this space can be represented as, $\underline{x} = \{x_1\}$

And x_1 can take n values, so the continuous space of $\underline{x}$ (line) has been discretized to a line with n grid points.

The probability mass value of each grid point is obtained as, $p(x_1=1), \dots, p(x_1=n)$ and so on.

Note: Again, the values that x_1 takes aren't in the range $(1, n)$, this is just for illustration purposes. But it is guaranteed that it will have n unique values.

If $d = 2$, then this space can be represented as, $\underline{x} = \{x_1, x_2\}$

Now both x_1 and x_2 can take n values in range, so the 2D continuous space has been discretized to a 2D space with n^2 grid points.

The probability mass value of each grid point is obtained as, $p(x_1=1, x_2=1), \dots, p(x_1=n, x_2=n)$ and so on.

So in general, for R^d continuous space, we will have a R^d space with n^d grid points.

We define a lattice in R^d space and let each grid point be a state in this lattice.

[More info on lattice in the first Lecture notes, Introduction section]

Construct a Markov Chain on the "state space" whose stationary distribution π is the target density $p(\underline{x})$ of interest.

That is, Perform a RW on the state space in such a way that the fraction of time spent in each state is proportional to $p(\underline{x})$

So in the Markov Chain that we create, each grid point i will be a state of the lattice/MC and associated with a coordinate $\underline{x}_i$ such that,

$$\pi_i = p(\underline{x}_i), i = 1, 2, \dots, n^d$$

So at this point we have found the π that we want to find the TPM for.

To get the TPM we have to generate a strongly connected directed graph of interest.

Each vertex x in this graph corresponds to a grid point of the R^d lattice.

So the graph has n^d vertices.

$\pi = p(\underline{x})$ for $\underline{x} \in V$ (grid points)

This basically converts a continuous distribution $p(\underline{x})$ into a discrete distribution π

Now we can say that the sequence of vertices in the RW after it converges to π is a good sample of the distribution $p(\underline{x})$

Actual Algorithm

Start with a connected undirected graph G on the set of states (vertices correspond to n^d grid points)

Then G is the lattice graph with $2 * d$ edges at each interior vertex, i.e. each vertex in G has edges to $2 * d$ of its neighbours (border vertices will not have all neighbours but don't worry about them here), if $d = 2$, then we get 2D case where each vertex has $2 * 2 = 4$ neighbours (top, bottom, left, right).

Let r be the max degree of any vertex in G .

Now we want to define a set of transitions from each vertex v in G , i.e., transitions from vertex i to vertex j which will give us P_{ij} which will fill the TPM P .

Transitions of the MC are defined as,

At state i ,

1. Select a neighbour j with probability $1/r$
2. Can also transition to itself (vertex i) with probability $1/r$

Suppose a neighbour j is selected (no self-transition)

1. If $\pi_j > \pi_i$, then go to vertex j .

What happens as a result of this is that points with higher probability masses are sampled (or visited by the RW) more.

Note that this doesn't take into consideration any other vertex that is connected to i or j , it is purely contained within two vertices i and j .

2. If $\pi_i > \pi_j$ then,

- a. Go to vertex j with probability π_j / π_i which has to be < 1 .

Because of this, points with lower probability masses are not as preferred by the RW.

- b. Remain at vertex i with probability $1 - \pi_j / \pi_i$

We can define self-transition probability as,

$$p_{ii} = 1 - \sum_{j=1, j \neq i}^n p_{ij}$$

We can combine all of this information to derive the equation for P_{ij}

$$P_{ij} = \frac{1}{r} \min\left(1, \frac{p_j}{p_i}\right) \implies (1)$$

$$\text{When } p_j > p_i, P_{ij} = \frac{1}{r} \cdot 1 = \frac{1}{r}$$

$$\text{When } p_j < p_i, P_{ij} = \frac{1}{r} \cdot \frac{p_j}{p_i}$$

We can multiply on both sides by p_i to get,

$$\begin{aligned} p_i \cdot P_{ij} &= \frac{p_i}{r} \min\left(1, \frac{p_j}{p_i}\right) \\ &= \frac{1}{r} \min(p_i, p_j) \\ &= \frac{p_j}{r} \min\left(\frac{p_i}{p_j}, 1\right) \\ &= p_j \cdot \left[\frac{1}{r} \min\left(\frac{p_i}{p_j}, 1\right) \right] \end{aligned}$$

$$p_i \cdot P_{ij} = p_j \cdot P_{ji} \implies (2)$$

This is called the **Condition of Detailed Balance**.

Flow of probability is maintained, i.e. whatever probability is going out is also coming in. P_{ij} is outgoing probability from i and P_{ji} is incoming probability to i .

And this condition holds true for each vertex in the graph, so for $N = n^d$ vertices, there will be N^2 edges in the graph and so N^2 equations representing Detailed Balance condition for each pair of edges.

We will now use this condition to move towards proving the fact that defining our transition probability via equation (1) actually would give us a RW that will converge to a stationary probability distribution, i.e., p_i and p_j belong to a stationary probability distribution.

Lemma: For a RW on a strongly connected directed graph, if the vector π satisfies the condition of detailed balance, i.e.,

$$\pi_x \cdot P_{xy} = \pi_y \cdot P_{yx} \text{ \& }$$

$$\sum_x \pi_x = 1$$

Then, π is a stationary probability distribution.

Proof:

$$\pi_x \cdot P_{xy} = \pi_y \cdot P_{yx}$$

Sum over y on both sides,

$$\sum_y \pi_x \cdot P_{xy} = \sum_y \pi_y \cdot P_{yx}$$

$$\pi_x \sum_y P_{xy} = \sum_y \pi_y \cdot P_{yx}$$

P is a row stochastic matrix because it is a TPM, so,

$$\sum_y P_{xy} = 1$$

Therefore,

$$\pi_x = \sum_y \pi_y \cdot P_{yx}$$

We can write this equation in matrix form as,

$$\pi_x = [\pi_1 \quad \dots \quad \pi_y \quad \dots \quad \pi_N] \cdot \begin{bmatrix} \dots & P_{1x} & \dots \\ \dots & P_{2x} & \dots \\ \dots & \dots & \dots \\ \dots & P_{Nx} & \dots \end{bmatrix}$$

[N = n^d]

So for computing π_x we multiply π with the xth column of P and if you remember the RW proofs that we did long back, this is the same equation as,

$$\pi = \pi \cdot P$$

Note: This equation is actually called the **Global Balance condition** and turns out it is a loose condition as compared to the Detailed Balance condition.

So Detailed Balance implies Global Balance

This basically proves that in equation (2), p_i and p_j belong to a stationary probability distribution and we can say that this is true for each vertex in the graph.

That is,

When the RW uses TPM as,

$$P = \{P_{ij}\}_{i,j=1\dots N}$$

$$P_{ij} = \frac{1}{r} \cdot \min\left(1, \frac{p_j}{p_i}\right)$$

Then,

$$\pi = [p_1 \quad \dots \quad p_i \quad \dots \quad p_j \quad p_N]$$

where, p_i = Discretized probability of $p(\underline{x})$ at $x = x_i$

And x_i = Real-vector associated with the grid point / vertex 'i' in the state graph / R^d lattice

So our choice of defining P_{ij} resulted in a TPM that ensures a RW will converge to the given stationary probability distribution $p(\underline{x})$ [discretized $p(\underline{x})$]

Note: Because n^d can be very large, instead of pre-computing TPM P, we can instead run a RW and depending on which vertices/grid points are visited, we can compute the ith row of P when we visit vertex i.

So we compute transition probabilities on the fly

And we're done with all the concepts here!

We divided the PDF into a discretized PMF which will be π and by using the Metropolis-Hasting algorithm found the TPM P .

We can then run a RW starting from any d-dimensional starting point following P as the TPM and let it converge to stationary probability distribution π

The grid points/vertices that have been visited in this RW would be the samples that follow the probability distribution π .

29/11/21

MODULE 4: Latent Variable Model

K-Means Algorithm

We have to cluster data which is unlabelled into K clusters which can be assigned cluster numbers of $\{1, 2, \dots, K\}$

Each data sample x_i is a d-dimensional data point

Therefore the problem statement becomes,

Given the data, $x_1, x_2, \dots, x_n$

Assign class labels to corresponding data points as, $y_1, y_2, \dots, y_n$

Such that,

$$y_i \in \{1, 2, \dots, K\}$$

How do we determine optimal labeling/clustering?

We have to define a metric such that points that have been assigned to the same cluster have high similarity and any points that have been assigned to different clusters will have low similarity.

01/12/21

Say that we have a function that takes in as input the clustering for a given set of data points and returns whether the given clustering is optimal or not via a metric that satisfies the above conditions.

Brute Force Approach

Say you have N data points and K possible classes/clusters

For each data point, you have K options.

Therefore for N data points, there are K^N ways of uniquely labelling the given data points.

So, give each unique clustering (K^T) to the optimization function to determine which clustering is the most optimal.

Clearly this is not so efficient as it's an NP algorithm.

Can you solve the labelling problem by some parameter estimation method?

Latent Variable

They are hidden inside the data/features (latent).

If the latent variables are known then they complete the data.

In this case, i.e. for the K-Means algorithm, the assigned class labels for the N data points, $\{y_1, y_2, \dots, y_N\}$ are the latent variables and we want to find them.

Define a cluster centre for each of the K clusters.

So there will be K cluster centres as,

$$\{\mu_1, \mu_2, \dots, \mu_K\}$$

μ_i = Cluster centre for the i^{th} cluster or class C_i

Note: Shape of μ_i = $d \times 1$, because it's a cluster centre in d -dimensional space

Initially we do not know anything about μ values because we don't know anything about which points are assigned to which clusters and so, we start with random cluster centres and points are also assigned to random clusters.

We want to find the optimal cluster centres and let this be modelled by a parameter θ such that,
 $\theta = [\mu_1 \quad \mu_2 \quad \dots \quad \mu_K]$

A cluster centre is said to be a good representation of the points in its cluster if all points in that cluster are "closest" to that particular cluster centre, i.e. intra-class distance

This can be measured via Euclidean distance.

So we can formulate the problem of finding θ as,

$$\theta^* = \arg \min(D(x, \theta))$$

$$\text{where, } D(x, \theta) = \sum_{i=1}^K \sum_{x_j \in C_i} d(x_j, \mu_i)$$

$d(x_j, \mu_i)$ = Euclidean Distance between point $x_j \in$ class C_i & cluster centre for class C_i , μ_i

So we want to find that θ (vector of cluster centres) which minimizes the sum of distances of each point j from its assigned cluster C_i

For more intuition we can write the same equation as,

$$D(x, \theta) = d(C_1) + d(C_2) + \dots + d(C_m)$$

where, $d(C_i)$ = Total distance of each point from its assigned class for given cluster centres θ

Therefore,

$\arg \min(D(x, \theta))$ returns that θ which gives

the minimum value for the overall sum of distances for each class

By minimizing this distance metric D via θ , you explicitly ensure that there will be high intra-class similarity / low intra-class distance but you also implicitly ensure that there will be low inter-class similarity / high inter-class distance.

Because the optimal cluster centres will be such that all points belonging to that cluster will be the nearest to its centre (minimizing intra-class distance) as compared to other cluster centres (maximizing inter-class distance).

So this distance metric D is such that it satisfied both our goals of inter and intra class distances and we didn't require optimizing two different metrics.

Gaussian Mixture Model (GMM)

Modelling a distribution (input data) as a mixture of Gaussians in different dimensions.

This allows the model to have more expressive power as compared to the base distribution (in this case, Gaussian) as we are allowed to play around with various Gaussians to fit multi-modal distributions (multiple peaks in the Gaussian curve) and reach higher dimensions.

Note: Gaussian refers to Multivariate Gaussian in this case, so a Gaussian is described using its Covariance matrix and mean vector, i.e., (μ_i, Σ_i) for i -th Gaussian

Recall that the formal notation for this is, $N(\mu_i, \Sigma_i)$

To map GMM to our clustering problem, we use one Gaussian for each cluster/class that we want/have.

When we mix many such Gaussians together, we assign a weight to each one of them while forming the Gaussian Mixture in order to indicate the importance of each Gaussian in the overall mixture.

This is called the Mixture coefficient and for the i -th Gaussian, it is denoted as, α_i

The coefficients have to be such that,

$$0 \leq \alpha_i \leq 1 \text{ \& } \sum_{i=1}^M \alpha_i = 1$$

Let θ = GMM Parameters, i.e. mean vector, covariance matrix and mixture coefficient of each Gaussian in the GMM.

$$\theta = \{(\mu_i, \Sigma_i, \alpha_i) \text{ for } i=1 \text{ to } M\}$$

Now we can define the conditional PDF for a data point x (d -dimensional vector) as,

$$P(x|\theta) = \sum_{i=1}^M \alpha_i \cdot N(x|\mu_i, \Sigma_i)$$

Recall,

$$N(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}} |\Sigma|^{-1/2} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

Which is the probability that data point x belongs to given Gaussian Distribution

Therefore,

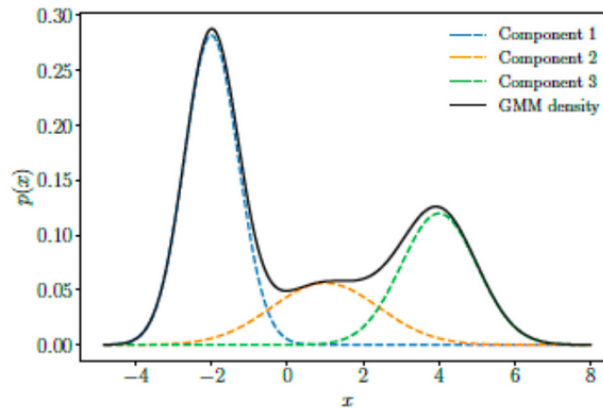
$P(x | \theta) \implies$ Probability that point x belongs to given GMM distribution

Example of a GMM modelling output,

350

Density Estimation with Gaussian Mixture Models

Figure 11.2
Gaussian mixture model. The Gaussian mixture distribution (black) is composed of a convex combination of Gaussian distributions and is more expressive than any individual component. Dashed lines represent the weighted Gaussian components.



components and the mixture density, which is given as

$$p(x|\theta) = 0.5\mathcal{N}(x|-2, \frac{1}{2}) + 0.2\mathcal{N}(x|1, 2) + 0.3\mathcal{N}(x|4, 1). \quad (11.5)$$

Ref: Marc Peter MML-book (Ch-11.1)

Here we represent the GMM density distribution (black curve) as a mixture of 3 Gaussians (blue, orange and green curves) whose parameters are as follows,

$$\theta = ((-2, 1/2, 0.5), (1, 2, 0.2), (4, 1, 0.3))$$

Using GMMs for clustering task

A GMM's parameters consist of the mean vector, covariance matrix and mixture coefficient for each class/cluster.

Whereas in our clustering task, we could just find the mean vector for each class/cluster.

So there's two approaches, in one we just find the mean vector and the other case is the more general one where we find the Covariance matrix as well as mixture coefficient.

The general approach clearly allows us more flexibility in fitting the data points onto the GMM distribution and the mean vector approach does also work but not as optimally on varied datasets.

Under what conditions does a GMM boil down to finding only the mean vector?

When all the mixture coefficients are equal, i.e. all classes have the same weightage, i.e. $1/M$ and all the covariance matrices are identity matrices, i.e. correlation between any two variables in each multivariate Gaussian added in this GMM is zero and the variance of each variable in each multivariate Gaussian added in this GMM is 1.

Now in this case we only have to find the optimal θ value which just consists of the mean vectors like before using this GMM model.

For both cases we can use Max. Likelihood Estimation.

MLE for finding theta

Find θ^* such that likelihood of $\underline{x}$ given θ is maximized.

where, $\underline{x} \implies$ Entire dataset of points $\implies n \times d$ matrix

$$\theta^* = \arg \max_{\theta} (P(\underline{x}|\theta))$$

If the data points are drawn using iid condition (independent and identically distributed random variables) from an unknown distribution $P(\underline{x})$ then we can rewrite the equation as,

$$\theta^* = \arg \max_{\theta} \left(\prod_{i=1}^n P(x_i|\theta) \right)$$

x_i = Point drawn from unknown GMM distribution

$$x_i \sim \sum_{j=1}^M \alpha_j \cdot N(x_i|\mu_j, \Sigma_j)$$

Property used: Joint probability distribution is the product of marginal distribution of each point if iid condition is followed.

Like we did before, this derivation can be simplified if we use log-likelihood and because log is a monotonically increasing function, maximizing log of RHS for θ is the same as maximizing S for θ

$$\begin{aligned}\theta^* &= \arg \max_{\theta} \left(\ln \left(\prod_{i=1}^n P(x_i|\theta) \right) \right) \\ &= \arg \max_{\theta} \left(\sum_{i=1}^n \ln (P(x_i|\theta)) \right)\end{aligned}$$

How does the task of maximizing the log likelihood for finding θ give us the same result as minimizing the distance metric for all clusters and for all data points to find θ ?

Note that for the i^{th} data point and the j^{th} class,

$$P(x_i|\theta) = \alpha_i \cdot N(x_i|\mu_j, \Sigma_j)$$

Now we can write each Multivariate Gaussian in the GMM for each data point x_i and j^{th} class C_j as,

$$N(x_i|\mu_j, \Sigma_j) = \frac{1}{(\sqrt{2\pi})^d |\Sigma_j|^{1/2}} \exp \left(-\frac{1}{2} \cdot (x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j) \right)$$

$$\text{let, } \frac{1}{(\sqrt{2\pi})^d |\Sigma_j|^{1/2}} = K_1, \frac{1}{2} = K_2, (x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j) = d^2(x_i, \mu_j, \Sigma_j)$$

Now equation becomes,

$$N(x_i|\mu_j, \Sigma_j) = K_1 \cdot \exp (-K_2 \cdot d^2(x_i, \mu_j, \Sigma_j))$$

How could we make the following assertion where $d()$ is the Euclidean distance metric and d^2 is the squared Euclidean distance?

$$(x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j) = d^2(x_i, \mu_j, \Sigma_j)$$

So if we maximize the likelihood, then we have to minimize the $d()$ term which is basically minimizing the intra-class distance. And this is how the two approaches lead to the same outcome.

Consider the special case where all covariance matrices have to be identity matrices.

Now consider that we are in 2D space and we are computing this equation for class number j for the i^{th} point, then,

$$(x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j) = [x_{i1} - \mu_{j1} \quad x_{i2} - \mu_{j2}] \cdot I_{2 \times 2}^{-1} \cdot \begin{bmatrix} x_{i1} - \mu_{j1} \\ x_{i2} - \mu_{j2} \end{bmatrix}$$

Clearly this computes to be,

$$= (x_{i1} - \mu_{j1})^2 + (x_{i2} - \mu_{j2})^2$$

And this is literally the equation for squared Euclidean distance of point x_i from mean vector of class j μ_j

Now consider the general case, i.e. covariance matrix was not the identity matrix, then,

$$d(x_i, \mu_j, \Sigma_j) = (x_i - \mu_j)^T \Sigma_j^{-1} (x_i - \mu_j)$$

This distance metric is called the **Squared Mahalanobis Distance** which is an **effective multivariate distance metric that measures the distance between a point and a distribution**.

Suppose that covariance matrix was a diagonal matrix, then,

$$\Sigma_j = \begin{bmatrix} \sigma_{j1}^2 & 0 & 0 \\ 0 & \sigma_{j2}^2 & 0 \\ 0 & 0 & \sigma_{jd}^2 \end{bmatrix}_{d \times d}$$

$$d(x_i, \mu_j, \Sigma_j) = [x_{i1} - \mu_{j1} \quad \dots \quad x_{id} - \mu_{jd}] \begin{bmatrix} \frac{1}{\sigma_{j1}^2} & 0 & 0 \\ 0 & \dots & 0 \\ 0 & 0 & \frac{1}{\sigma_{jd}^2} \end{bmatrix} \begin{bmatrix} x_{i1} - \mu_{j1} \\ \dots \\ x_{id} - \mu_{jd} \end{bmatrix}$$

$$= \sum_{k=1}^d \frac{(x_{ik} - \mu_{jk})^2}{\sigma_{jk}^2}$$

This is called the **Squared Weighted Euclidean Distance** (where variances are the weights on each distance term)

Now,

$$P(x_i|\theta) = \text{Likelihood of datapoint } x_i \text{ to come from GMM, i.e., } \theta$$

$$= A + B + C + \dots$$

where,

$$A = \alpha_1 \cdot N(x_i|\mu_1, \Sigma_1)$$

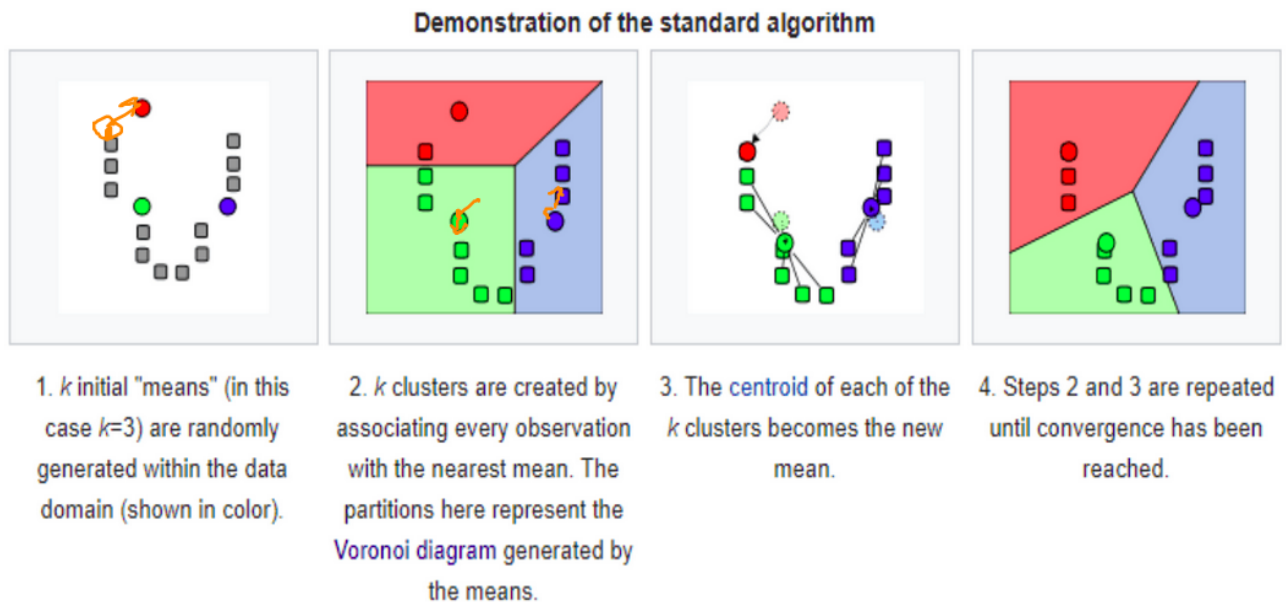
$$B = \alpha_2 \cdot N(x_i|\mu_2, \Sigma_2)$$

$$C = \alpha_3 \cdot N(x_i|\mu_3, \Sigma_3)$$

...

In the MLE approach, to find θ^* , we have to solve the log-likelihood equation which is a combination of Gaussian distributions and this is not easy to do analytically because of complicated differentiations and integrations, so we naturally move on to the iterative approach.

K-Means: Only Mean Vectors as Parameters



K-Means: Using GMM Parameters

[Assume that we have M classes/clusters and T data points in d -dimensional space]

Step 1: Initialization

We initialize all parameters of θ randomly, i.e. mean vectors, covariance matrices and mixture coefficients.

$$\mu_1^{(1)}, \mu_2^{(1)}, \dots, \mu_M^{(1)}$$

$$\Sigma_1^{(1)}, \Sigma_2^{(1)}, \dots, \Sigma_M^{(1)}$$

$$\alpha_1^{(1)}, \alpha_2^{(1)}, \dots, \alpha_M^{(1)}$$

This constitutes $\theta^{(1)}$

Note:

- The superscript (1) represents the 1st iteration and similarly we'll continue the process for many iterations.

-

$$\mu_1 \implies \text{dx1 size}$$

$$\Sigma_1 \implies \text{dxd size}$$

$$\alpha_1 \implies \text{Constant}$$

Step 2: Optimal Partitioning

[Assigning data points to clusters]

Two ways to do this, Soft Partitioning and Hard Partitioning

Hard Partitioning vs Soft Partitioning: Overview

The main difference between the two can be described as follows,

The output of the Partitioning task is the cluster that each data point has been assigned to.

In the case of Hard Partitioning, each data point has to belong to exactly one class/cluster. So it is a deterministic partitioning and the cluster assigned to a data point can be represented by an indicator vector as,

$[1 \ 0 \ \dots \ 0 \ 0] \implies$ Data point belongs to class 1

In the case of Soft Partitioning, each data point can belong to more than one class/cluster. So it is non-deterministic partitioning and the cluster/clusters assigned to a data point can be represented by a PMF as,

$[0.3 \ 0.2 \ \dots \ 0 \ 0]$

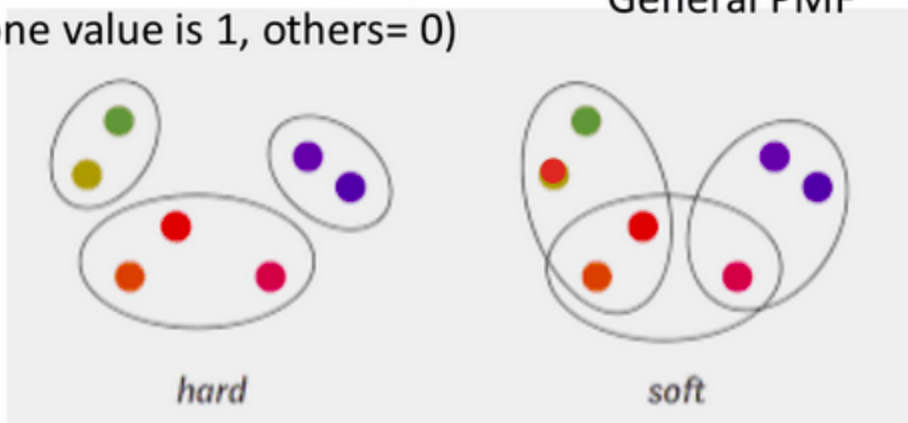
The data point has a 30% chance of belonging to class 1, 20% chance of belonging to class 2 and so on.

Ofcourse the sum of a PMF has to be 1 is to be noted.

This image should give you more intuition into what is happening

Indicator function (only
one value is 1, others= 0)

General PMF



Step 2.a: Soft Partitioning

Find $P(i|x_t, \theta^{(n)})$

This term represents the responsibility of cluster i for the data point x_t or in other words, how likely is class i given x_t

It is the posterior probability of cluster i given the data point x_t and the GMM parameters for n^{th} iteration.

This can be computed as,

$$\begin{aligned}\gamma_t(i) &= P(i|x_t, \theta^{(n)}) \\ &= \frac{P(i, x_t, \theta^{(n)})}{P(x_t, \theta^{(n)})} \implies \text{Using Bayes Rules (Note: } , = \cap \text{)} \\ &= \frac{P(x_t|i, \theta^{(n)}) \cdot P(i, \theta^{(n)})}{P(x_t, \theta^{(n)})}\end{aligned}$$

$P(x_t|i, \theta^{(n)}) \implies$ Probability that x_t comes from i^{th} class's Gaussian distribution in GMM which has parameters, $\mu_i^{(n)}, \Sigma_i^{(n)}$

This can directly be written as,

$P(x_t|i, \theta^{(n)}) = N(x_t|\mu_i^{(n)}, \Sigma_i^{(n)}) \implies$ Represents likelihood that we were maximizing

$P(i, \theta^{(n)}) \implies$ Probability that the i^{th} class is chosen from $\theta^{(n)}$

This is literally just α_i

$P(x_t, \theta^{(n)})$ is independent of i and so will not factor a role in the optimization process

Therefore, maximizing the a-posteriori probability will also end up maximizing the likelihood so we'll go ahead with this task instead of the MLE task.

$\gamma_t(i) \implies$ Responsibility of cluster i for data point t

$\gamma_t \implies$ Vector which shows responsibility of each cluster for t

$$\gamma_t = [\gamma_t(1) \quad \gamma_t(2) \quad \dots \quad \gamma_t(i) \quad \dots \quad \gamma_t(M)]_{1 \times M}$$

This is the PMF for a given point x_t that was described in the overview section.

Here we can do partitioning by considering the posteriori probability PMF and picking a cluster from it for a given point, like 30% chance of point being from class 1, 20% chance of point being class 2 and so on. The partitioning is not required for updating parameters as is the case with Hard Clustering, however we do need it in the next step even for Soft Clustering which is the Convergence Check.

Step 2.b: Hard Partitioning

For each point x_t apply Maximum a posteriori rule (**MAP Rule**) to find the class/cluster label y_t for it such that, $y_t \in \{1, 2, \dots, M\}$

[Note: It is called a posteriori because we already know information about point x_t]

$$\begin{aligned} y_t &= \arg \max_i \left(P(i|x_t, \theta^{(n)}) \right) \\ &= \arg \max_i \left(\frac{N(x_t|\mu_i, \Sigma_i) \cdot \alpha_i}{P(x_t, \theta^{(n)})} \right) \\ &= \arg \max_i (N(x_t|\mu_i, \Sigma_i) \cdot \alpha_i) \implies \text{Denominator independent of } i \\ &= \arg \max_i (K_1 \cdot \exp(-K_2 \cdot d(x_t, \mu_i, \Sigma_i))) \implies \text{Already proven before} \end{aligned}$$

where, $d(x_t, \mu_i, \Sigma_i)$ = Mahalanobis/Squared Euclidean/Weighted Euclidean Distance

In summary, each point is assigned to one and only one cluster which gives maximum a-posteriori probability.

Now we can define the partitions for each class as,

Partition for class i , P_i consists of those points for which class i gave maximum a-posteriori probability

$$\begin{aligned} P_i &= \{x_t : \gamma_t(i) > \gamma_t(j), \forall j \neq i\} \\ &= \{x_t : \alpha_i \cdot N(x_t|\mu_i, \Sigma_i) > \alpha_j \cdot N(x_t|\mu_j, \Sigma_j)\} \end{aligned}$$

Step 3: Update Parameters

Step 3.b: For Hard Clustering

[Soft Clustering defined later as it's a bit more complicated]

Defining indicator vector using gamma vector as,

$$\begin{aligned} \gamma_t(i) &= 1 \text{ if } y_t = i \\ &= 0 \text{ otherwise} \end{aligned}$$

So i^{th} entry for x_t is 1 only if x_t has been assigned to i^{th} class/cluster

i) Updating mean vectors,

Mean vector for class i for next iteration (updating step) will be the mean of all points that were assigned to class i in Step 2.b

This can be written as,

$$\mu_i^{(n+1)} = \frac{\sum_{x_t \in P_i^{(n)}} x_t}{|P_i^{(n)}|}$$

where, $P_i^{(n)}$ = Set of points that were assigned to i^{th} cluster in n^{th} iteration

In short, this is just the sample mean of $P_i^{(n)}$

ii) Updating Covariance matrices,

Covariance matrix for class i for next iteration will be the mean of the covariance matrices of all points that were assigned to class i in Step 2.b

This can be written as,

$$\Sigma_i^{(n+1)} = \frac{\sum_{x_t \in P_i^{(n)}} (x_t - \mu_i^{(n+1)}) (x_t - \mu_i^{(n+1)})^T}{|P_i^{(n)}|}$$

Numerator computes the sum of Covariance matrices of each data point that has been assigned to class i

In short, this is just the sample covariance matrix of $P_i^{(n)}$

iii) Updating Mixing coefficients,

Updated mixing coefficient for class i is the ratio of the number of points that were assigned to class i divided by the total number of points and this makes sense because alpha represents the importance of any particular class, the more points are assigned to the same class the greater its importance will be.

$$\alpha_i^{(n+1)} = \frac{|P_i^{(n)}|}{T}$$

Step 3.a: For Soft Clustering

i) Updating mean vector

$$\mu_i^{(n+1)} = \frac{\sum_{t=1}^T \gamma_t(i) \cdot x_t}{\sum_{t=1}^T \gamma_t(i)}$$

We do not know to exactly which cluster does a point belong to, so there's no concept of P_i

But we know how likely a point x_t is to belong to cluster i, $\gamma_t(i)$

So instead of the sample mean, we compute the weighted sample mean with a-posteriori probabilities as the weights over all the points in the data (T data points).

ii) Updating Covariance matrices

$$\Sigma_i^{(n+1)} = \frac{\sum_{t=1}^T \gamma_t(i) (x_t - \mu_i^{(n+1)}) (x_t - \mu_i^{(n+1)})^T}{\sum_{t=1}^T \gamma_t(i)}$$

Just like with the mean vectors, updated covariance matrices are the weighted sample covariance matrices where a-posteriori probabilities are the weights.

iii) Updating Mixing coefficients

$$\alpha_i^{(n+1)} = \frac{\sum_{t=1}^T \gamma_t(i)}{T}$$

This also directly follows from how we've updated mean vectors and covariance matrices.

Step 4: Convergence Check: Stopping Criteria

Have a **tolerance for error** epsilon, ϵ which is **very close to zero like 10^{-10}** .

Define distance metric as,

$$D^{(n+1)} = \frac{\sum_{i=1}^M \sum_{x_t \in P_i^{(n)}} d(x_t, \mu_i^{(n+1)}, \Sigma_i^{(n+1)})}{T}$$

Note that this uses, $P_i^{(n)}$ as the partition set because we are still in the n^{th} iteration only and the partition for the $(n+1)^{\text{th}}$ iteration hasn't happened yet.

And note that this is irrespective of whether we are doing Soft or Hard Clustering.

This metric is measuring the **average intra-class distance after n^{th} iteration**

$$\text{If } \frac{|D^{(n+1)} - D^{(n)}|}{D^{(n)}} \leq \epsilon, \text{ then}$$

Stop with $\theta^{(n+1)}$ as the solution, i.e.

$\theta^* = \theta^{(n+1)}$ and $P^{(n+1)} = \text{Optimal clustering}$

Else,

$n = n + 1$ and go to step 2

The K-Means Algorithm guarantees finding of a local minima, however there is no guarantee of finding the global minima.

The algorithm can end up finding the global minima, if for the initial parameter values, the local minima is the same as the global minima.

So, the setting of the initial parameter values can play a big role in finding more optimal solutions.

06/12/21

How does K-Means guarantee Local Optimum?

There was a very involved proof method taught by ma'am but overall it seemed quite pointless so I just didn't write it :(

The basic essence is that we can guarantee that after every parameter update step, the total intra-class distance, i.e. $D^{(n+1)}$ will be less than or equal to the total intra-class distance before the parameter update step but after the partitioning step, after Step 2 and before Step 3, i.e. $D^{(n)}$

and this will be less than or equal to the total intra-class distance partitioning step, i.e. before Step 2, i.e. $D^{(n)}$

So every time we update the parameters, we are guaranteeing that the total intra-class distance will be lesser than the total intra-class distance we had before and so we are guaranteed to reach a local minimum at which point our tolerance criteria will be satisfied and K-Means converges.

This is more of an intuitive way to understand and definitely not a formal proof of convergence but the original proof didn't seem like much of a proof in the first place, so it is what it is.

<https://shabal.in/visuals/kmeans/1.html>

Link for visualizing how different starting values for cluster centers can result in completely different convergence of clusters, i.e. various local minima and not global minima.

An approach for finding the global minima is that you could run K-Means with different starting cluster centres many many times and finally pick that set of initial cluster centres which gives minimum intra-class distance at convergence, basically, **this works on the idea that minimum of enough local minimums will give you global minima.**

08/12/21

Expectation Maximization Algorithm (EM)

Intro

The given task:

Given data x consisting of T data points in d -dimensional space, i.e.

$x = (x_1, x_2, \dots, x_T)$ & $x_1, x_2, \dots, x_T = dx1$ shape

Find the best fitting PDF, $P(x|\theta)$ in a specific form, parameterized by θ

In this case, the θ need not be parameters to a GMM, it can be just a Gaussian distribution, Uniform Distribution, etc. or even just a mean vector, i.e. what is the probability that x comes from a distribution which has mean vector θ

Because Gaussians are highly applicable to ML tasks and have analytically tractable solutions as well as properties, we will go ahead with considering Gaussians for this discussion.

In short, Find,

$$\theta^* = \arg \max_{\theta} (P(x|\theta))$$

We can use MLE to solve this problem.

If the points in x are drawn with iid assumption then,

$$P(x|\theta) = \prod_{t=1}^T P(x_t|\theta)$$

As before, taking log to get log Likelihood will help simplify calculations, therefore,

$$\begin{aligned} l(\theta) &= \ln (P(x|\theta)) \\ &= \ln \left(\prod_{t=1}^T P(x_t|\theta) \right) \\ &= \sum_{t=1}^T \ln (P(x_t|\theta)) \end{aligned}$$

We can solve this analytically, i.e. closed-form solution as,

$$\text{Set, } \frac{\partial l(\theta)}{\partial \theta} = 0 \implies \text{If } \theta = \text{Scalar}$$

$$\text{Set, } \nabla_{\theta} l(\theta) = 0 \implies \text{If } \theta = \text{Vector}$$

Note: The grad function, i.e. ∇_{θ} denotes that $l(\theta)$ will be differentiated by each element of θ and the differentiation will be set to zero.

Consider the case that we have a Single Univariate Gaussian, so data is 1D and mean and variance are also 1D, i.e.,

$$\begin{aligned} P(x_t|\theta) &= N(x_t|\mu, \sigma^2) \\ &= \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{1}{2\sigma^2} (x_t - \mu)^2 \right) \end{aligned}$$

Let,

$$K_1 = \frac{1}{\sqrt{2\pi\sigma^2}}, K_2 = \frac{1}{2\sigma^2}$$

Therefore,

$$\begin{aligned} l(\theta) &= \sum_{t=1}^T \ln \left(K_1 \cdot \exp \left(-K_2 \cdot (x_t - \mu)^2 \right) \right) \\ &= T \cdot \ln (K_1) - K_2 \sum_{t=1}^T (x_t - \mu)^2 \end{aligned}$$

Now we can easily set differentiation of this to zero wrt mean

$$\frac{\partial l(\theta)}{\partial \mu} = 0$$

$$-K_2 \cdot 2 \cdot (-1) \sum_{t=1}^T (x_t - \mu^*) = 0$$

$$2K_2 \sum_{t=1}^T (x_t - \mu^*) = 0$$

$$\sum_{t=1}^T x_t = T \cdot \mu^*$$

$$\mu^* = \frac{\sum_{t=1}^T x_t}{T}$$

So the optimal mean value is the sample mean of the T points in x

Similarly we can repeat the same steps for variance to get,

$$(\sigma^2)^* = \sum_{t=1}^T \frac{(x_t - \mu)^2}{T}$$

So the optimal variance value is the sample variance of the T points in x

And similarly we can also prove that, for Multivariate Gaussian case, i.e. $d > 1$, the optimal mean and Covariance matrices will be the sample mean of all the data points and Sample Covariance matrices of all the data points, respectively.

$$\mu^* = \frac{1}{T} \sum_{t=1}^T x_t$$

$$\Sigma^* = \frac{1}{T} \sum_{t=1}^T (x_t - \mu) \cdot (x_t - \mu)^T$$

Problems with MLE Approach

For most problems like the K-Means case where the PDF was a GMM, such analytical derivations with closed-form solutions are not possible.

To avoid this we need to use some other iterative technique and that's the EM Algorithm.

Base Concepts for EM Algorithm

A general method for finding the Maximum Likelihood Estimate of the parameters of an underlying distribution from a given dataset.

Useful when the data is incomplete/has missing values.

Example, feature components are missing or labels are missing/not available.

Case 1:

d-dimensional vectors have missing values due to limitations of the observation process.

So there's missing data in the features of the dataset, i.e. **feature extraction from noisy data**

Case 2:

Optimizing $l(\theta)$ is not analytically possible but $l(\theta)$ can be simplified by assuming the existence of additional missing or hidden data.

[The name Latent variables should be ringing bells right about now.]

Case 1 is not of much concern for our syllabus and we most just focus on case 2 and its applications.

What does the notion of missing or hidden data mean?

Assume that you are given a set of T data points x, however you know that this is not the complete set of data.

There is another unknown set of data y which when combined with x gives you the complete data z. The only thing we know about y is that it's unknown, random and governed by an underlying probability distribution which depends on the specific problem being solved.

For example, the underlying probability distribution can be a GMM when you have class labels y in the data.

Explanation: The class labels y are unknown and the data points x are known in the case of a clustering problem, i.e. use K-Means algorithm, so in it we have GMM as the underlying probability distribution as we have seen before.

[Note: We are given this underlying probability distribution while doing the EM Algorithm.]

In short, missing data refers to the class labels in case of unsupervised learning tasks and can refer to the target variable of the test data in case of supervised learning tasks.

Expanding on this further,

For the clustering task, say we have M classes and T data points.

So each data point in x can be assigned to any one of the M classes.

So in total there can be M^T total possible assignments.

Assigning each point to a class will give you a vector of assignments of size $1 \times T$ which is actually the cluster assignments "y" and so in total **there will be M^T such y vectors and this collection of y vectors constitutes the latent variable space Y.**

So the hidden data y is a random variable such that, $y \in Y$

Let

$z = (x, y) \implies \text{COMPLETE DATA}$

And we want to compute the complete data likelihood, i.e. joint density, i.e.,

$$P(z|\theta) = P(x, y|\theta)$$

[Notation Change: Iterations are indicated using i instead of n now]

And like before we take the log of this and find the complete data log likelihood, i.e.

$$\ln(P(x, y|\theta))$$

wrt the unknown data y and given the observed data x and the current parameter estimates of

$$\theta \implies \theta^{(i)}$$

Remember that we added this hidden variable concept into our typical K-Means/clustering problem in order to find,

$$\theta^* = \arg \max_{\theta} (P(x|\theta))$$

We want to find the max. log likelihood of the complete data however we only have access to x data and current parameter value $\theta \implies \theta^{(i)}$

We imposed additional hidden variable y which we actually know nothing about, so we have to marginalize it from the complete data log likelihood which can be done by averaging it out from the likelihood, this adds an Expectation component to the log likelihood as,

$$E_{y|x, \theta^{(i)}} \left(\ln(p(x, y|\theta)) | x, \theta^{(i)} \right)$$

This looks absolutely nuts but we are basically computing the Expectation of the log-likelihood of the complete data given data x and $\theta^{(i)}$

So we are trying to estimate the distribution of y given x and $\theta^{(i)}$

This is very very similar to the posterior probability that we used in K-Means algorithm which computed,

$$\gamma_t(i) = P(i|x_t, \theta^{(n)})$$

So posterior probability was the estimation of distribution of y in the K-Means case.

Hopefully this is a good enough mapping to understand what we are doing.

The EM algorithm defines a new quantity called the **EM Auxiliary Function**,

$$Q(\theta, \theta^{(i)}) = E_{y|x, \theta^{(i)}} \left(\ln(p(x, y|\theta)) | x, \theta^{(i)} \right) \implies (1)$$

where,

$\theta^{(i)}$ = Current parameter estimate that we use to evaluate expectation

θ = New parameter to be optimized to increase $Q(\theta, \theta^{(i)})$

And it can be proved that,

$$Q(\theta, \theta^{(i)}) = \text{Lower Bound on } \ln(P(x|\theta))$$

Which is exactly what we want to find!

Note:

$x, \theta^{(i)}$ are given and y is a random variable governed by the distribution

$$P(y|x, \theta^{(i)})$$

i.e., given data x & $\theta^{(i)}$ induces a distribution for y in Y space.

Note: Some y are more likely than others given current parameters.

The RHS of equation (1) can be written as,

$$E_{y|x, \theta^{(i)}}(\ln(p(x, y|\theta))|x, \theta^{(i)}) = \int_{y \in Y} \ln(P(x, y|\theta)) \cdot P(y|x, \theta^{(i)}) dy$$

[Expectation over a continuous distribution is an integration and we are marginalizing the integration over y , so what is left will be a function of x and θ]

[If y was a discrete random variable, i.e. Y is a discrete distribution, in that case, the integration will be replaced by a summation over all discrete values of y]

Note that

$$P(y|x, \theta^{(i)})$$

is the marginal distribution of unobserved data and is dependent on both, the observed data as well as the current parameters.

This is also a known quantity, we may not know the class labels/unknown data y , but we know its distribution.

Note that x doesn't depend on θ , so we can also write the marginal distribution as,

$$P(y, x|\theta^{(i)})$$

But it is not necessary that we always do this, it is only used in the situation that we cannot compute the original marginal distribution described before.

Algorithm Steps

E-Step (Expectation):

$$Q(\theta, \theta^{(i)}) = \int_{y \in Y} \ln(P(x, y|\theta)) \cdot P(y|x, \theta^{(i)}) \implies (A)$$

M-Step (Maximization):

$$\theta^{(i+1)} = \arg \max_{\theta} (Q(\theta, \theta^{(i)})) \implies (B)$$

These two steps are iteratively repeated until convergence of θ is obtained.

Each iteration is guaranteed to increase the log likelihood and the algorithm is guaranteed to converge to a local maximum of the likelihood function.

09/12/21

Do note that we are doing all of this to maximize log likelihood in situations where we can't do so analytically, so the iterative approach involves maximizing this EM Auxiliary function Q which will be guaranteed to be the lower bound on the actual log likelihood.

The points at which the value of Q actually becomes equal to the actual log likelihood are called **points of tangency**.

That is,

If $l(\theta) = Q(\theta, \theta^{(i)}) \implies \theta^{(i)}$ is point of tangency

Proof for EM Algorithm:

Again a very involved proof that involves something called Jensen's Inequality to prove that Q function is the lower bound on the actual log likelihood but it was quite annoying to follow so I'm just leaving it as it is :(

All you need to know from application POV is the EM algorithm that was described before and the motivation for why we're using this in the first place which is explained in short in the previous section.

FOR EXAMS

Numericals from MCMC and PageRanking and everything before K-Means is possible. But from K-Means onwards just try to understand the concepts for the exam.